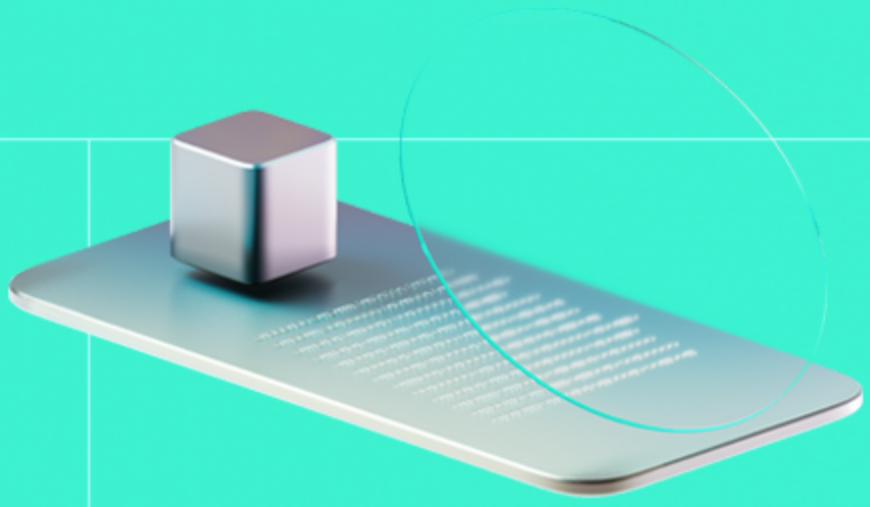
HACKEN

Smart Contract Code Review And Security Analysis Report



Customer: Tokenize.it

Date: 15/05/2026

We express our gratitude to the Tokenize.it team for the collaborative engagement that enabled the execution of this Smart Contract Security Assessment.

The tokenize.it protocol is a modular smart contract system for tokenizing real-world company equity on the EVM. It provides a complete lifecycle — from token issuance with compliance enforcement, through primary and secondary market sales, vesting, dividend distribution, and exit — with platform-level fee collection and an `AllowList`-based KYC/attribute registry governing who may transact.

Document

Name	Smart Contract Code Review and Security Analysis Report for Tokenize.it
Audited By	Kornel Światłowski, Khrystyna Tkachuk
Approved By	Kerem Solmaz
Website	https://tokenize.it/en
Changelog	01/05/2026 - Preliminary Report
	15/05/2026 - Final Report
Platform	Ethereum
Language	Solidity
Tags	Fungible Token, Permit Token, Meta Transactions, Vesting, Marketplace, Vault, Claims
Methodology	https://docs.hacken.io/methodologies/smart-contracts

Review Scope

Repository	https://github.com/corpus-io/tokenize.it-smart-contracts.git
Commit	4414c631a341dfed79611d040ae0f5aa9801243f
Final Commit	52b0322fb566c7143d09c23b7bd30f2e092e0691

Audit Summary

The system users should acknowledge all the risks summed up in the risks section of the report

22	21	1	0
Total Findings	Resolved	Accepted	Mitigated

Findings by Severity

Severity	Count
Critical	0
High	0
Medium	2
Low	9

Vulnerability	Severity	Status
F-2026-16556 - Push Based Settlement With Immutable Recipients Enables Permanent Denial Of Service	Medium	Fixed
F-2026-16595 - Inconsistent Validation of LockedUntil Enables Premature Privileged Actions	Medium	Fixed
F-2026-16480 - Missing Base Price Validation In Initialization Enables Full Profit Redirection	Low	Fixed
F-2026-16481 - Currency Update Without Token Price Sync Leads To Mispricing	Low	Fixed
F-2026-16558 - Privileged Burn Function Allows Arbitrary Token Destruction	Low	Accepted
F-2026-16606 - Lack of Exit Token Validation Allows Mismatched Contract Binding	Low	Fixed
F-2026-16644 - Asymmetric Parameter Update Logic Allows Immediate Price Availability	Low	Fixed
F-2026-16661 - Max Purchase Limit Is Applied Per Receiver Instead of Per Buyer	Low	Fixed
F-2026-16745 - Vesting.commit Overwrites Existing Commitment State, Including Revoked or Already-Revealed Hashes	Low	Fixed
F-2026-16771 - Zero Price Allows Free Token Acquisition and Causes Division-by-Zero	Low	Fixed
F-2026-16772 - Missing Zero-Address Validation for _owner in PayoutBase May Lead to Loss of Administrative Control	Low	Fixed
F-2026-16482 - Single Step Ownership Model Enables Immediate Loss Of Administrative Control	Info	Fixed

Vulnerability	Severity	Status
F-2026-16638 - Public Function Not Used Internally Can be Marked External	Info	Fixed
F-2026-16639 - Lack of Event Emitting for Important State Changes	Info	Fixed
F-2026-16641 - Repeated Storage Reads of leadInvestors.length in a Loop Condition	Info	Fixed
F-2026-16656 - Mismatched Cooldown Period Between Implementation and Invariants Breaks Assumptions	Info	Fixed
F-2026-16657 - Token Fee-Collector Compliance Bypass Applies Only to Mints Instead of All Transfers	Info	Fixed
F-2026-16741 - Missing State Cleanup After Fee Settings Acceptance	Info	Fixed
F-2026-16743 - Missing Self-Reassignment Validation	Info	Fixed
F-2026-16768 - Gas Inefficiency Due to Redundant Boolean Equality Check	Info	Fixed
F-2026-16769 - Redundant coolDownStart Assignment in activateDynamicPricing	Info	Fixed
F-2026-16770 - Missing Inherited Initializer Calls in Clone Contracts	Info	Fixed

Documentation quality

- Functional requirements are well documented.
  - The project overview is detailed.
  - All roles within the system are clearly described.
  - Use cases are documented and explained in detail.
  - All features of each contract are described.
  - Interactions between contracts are clearly outlined.
- The technical description is comprehensive.
  - Run instructions are provided.
  - NatSpec documentation is sufficient.
  - A technical specification is provided.

Code quality

- The development environment is configured.
- The codebase reuses multiple OpenZeppelin contracts and libraries.

Test coverage

Code coverage of the project is **100%** (branch coverage).

- Deployment and basic user interactions are covered with tests.
- Negative cases coverage is present.

Table of Contents

System Overview **7**

Privileged Roles 10

Potential Risks **15**

Findings

Vulnerability Details

Disclaimers 67

Appendix 1. Definitions **68**

Severities 68

Potential Risks 68

Appendix 2. Scope **69**

Appendix 3. Additional Valuables **71**

System Overview

The **tokenize.it** protocol is a modular smart contract system for tokenizing real-world company equity on the EVM. It provides a complete lifecycle — from token issuance with compliance enforcement, through primary and secondary market sales, vesting, dividend distribution, and exit — with platform-level fee collection and an `AllowList`-based KYC/attribute registry governing who may transact. It contains following contracts:

`Crowdinvesting` is a public token sale contract that allows anyone to purchase tokens at a configurable price, with one instance deployed per token. It supports both a static base price and dynamic pricing via an external `IPriceDynamic` oracle bounded by a price floor and ceiling. Tokens are either minted directly or transferred from a designated `tokenHolder`. The contract enforces per-receiver minimum and maximum purchase amounts, supports ERC677 (`transferAndCall`) in addition to standard ERC20 buy flow, and provides an optional auto-pause via a `lastBuyDate` timestamp. All parameter mutations require the contract to be paused and trigger a 1-hour cooldown before unpausing is allowed. The contract is deployed via clone/proxy pattern, uses Ownable2Step, Pausable, ReentrancyGuard, and supports ERC2771 meta-transactions. Fees are collected on every purchase through the token's `FeeSettings` contract, and the payment currency must be a `TRUSTED_CURRENCY` on the token's `AllowList`.

`FeeSettings` is the platform-level fee management contract for tokenize.it. Fee types (e.g., TOKEN, CROWDINVESTING, PRIVATE_OFFER, SECONDARY_MARKET, DISTRIBUTION, EXIT) are dynamically registered, each with a hard cap (`maxNumerator`), a default rate, and a default fee collector, with all numerators expressed out of a `FEE_DENOMINATOR` of 10,000. The owner can propose and execute default fee changes, with a 12-week delay enforced on any fee increase. Managers — a delegated role granted by the owner — can set per-token custom fee discounts (which can only reduce, never increase, the effective fee) and per-token custom fee collectors, each with an expiration date after which they revert to defaults. The contract implements `IFeeSettingsV1`, `IFeeSettingsV2`, and `IFeeSettingsV3` for backward compatibility, is deployed via clone/proxy pattern, and uses Ownable2Step, ERC165, and ERC2771 meta-transactions.

`Vesting` handles linear ERC20 token vesting for multiple beneficiaries, supporting two token custody models: the contract holds tokens directly (transferable), or tokens are minted on release (mintable). Vesting plans are created by managers and feature a configurable allocation, cliff period, and total duration — if the cliff exceeds the duration, the duration is automatically extended to match. Plans can be created transparently (public) or privately via a commit/reveal scheme where the hash is stored first and parameters revealed later; commitments can be revoked before reveal, which limits the vesting end date. Active vesting plans can be stopped (early terminated) or paused (stopped and replaced with a new plan with adjusted terms). The beneficiary claims vested tokens via the `release()` function and can transfer their beneficiary status to a new address; the owner can also change the beneficiary but only one year after the vesting plan ends. The contract uses auto-incrementing uint64 plan IDs, is deployed via clone/proxy pattern, and uses Ownable2StepUpgradeable, ReentrancyGuardUpgradeable, and ERC2771 meta-transactions. The owner is automatically a manager at initialization.

`Token` is a UUPS-upgradeable ERC20 implementing company tokenization for the tokenize.it platform. It extends the standard ERC20 with pausing, ERC20Snapshot for historical balance queries, EIP-2612

permit, and role-based access control with eight dedicated roles: `DEFAULT_ADMIN_ROLE`, `REQUIREMENT_ROLE`, `MINTALLOWER_ROLE`, `BURNER_ROLE`, `TRANSFERERADMIN_ROLE`, `TRANSFERER_ROLE`, `PAUSER_ROLE`, and `SNAPSHOTCREATOR_ROLE`. Minting is controlled by per-address minting allowances managed by the `MINTALLOWER_ROLE`, with automatic fee collection to the fee collector on every mint. The `BURNER_ROLE` can burn tokens from any address. Every transfer (excluding mints and burns) enforces that both sender and receiver satisfy a configurable requirements bitmask checked against the `AllowList`, unless either party holds the `TRANSFERER_ROLE`. Fee settings can be migrated through a two-step suggest/accept flow requiring both the current `FeeSettings` owner and the token's `DEFAULT_ADMIN_ROLE`. The contract uses a storage gap of 446 slots to anchor the `allowList` variable at slot 1000, supports ERC2771 meta-transactions, and uses 18 decimals.

`CoinvestedPosition` holds tokens and sells them at a preset price, distributing proceeds between a co-investor (owner) and lead investors whose carry shares are each set by a `profitFraction` (uint32). Lead investors receive carry only on profit above the base price; the co-investor receives the residual — base portion plus leftover profit on the upside, and the full proceeds (absorbing the entire downside) whenever proceeds do not exceed the base price. The contract extends `TokenSwapBase` and supports buying tokens via a `buy()` function, claiming dividends from `Distribution` contracts (where the full received amount is treated as profit and split accordingly), and claiming exit proceeds from `Exit` contracts with carry calculated on the surplus above an effective base price. The owner can change the payment currency and base price via `setCurrency()`, but both this function and `unpause()` are blocked until a `lockedUntil` timestamp has elapsed. The contract integrates with `GlobalTokenExitRegistry` for exit claiming, starts paused on initialization, and is deployed via clone/proxy pattern with Ownable2StepUpgradeable, PausableUpgradeable, ReentrancyGuardUpgradeable, and ERC2771 meta-transactions.

`Distribution` distributes proceeds such as dividends to token holders proportional to their balances at a specific ERC20Snapshot, using a snapshot ID from `Token.createSnapshot()` to determine each holder's share and a fixed `pricePerToken` to compute the currency payout. Holders claim their share via the `claim()` function, with fees deducted through `IFeeSettingsV3`. The owner can reassign unclaimed funds from one address to another after the `lockedUntil` timestamp (useful for holders who lost access to their address), and can drain remaining ERC20 balances after the same lock period. Initial reassignments can be provided at initialization, bypassing the time restriction. The contract extends `PayoutBase`, is deployed via clone/proxy pattern, and uses Ownable2Step, ReentrancyGuard, and ERC2771 meta-transactions.

`Exit` is an automated exit contract where token holders exchange their full token balance for currency at a fixed `pricePerToken`. Holders call `claim()`, transferring their tokens to the contract and receiving the currency payout minus fees (deducted via `IFeeSettingsV3`). The received tokens are held in the contract and can be burned or extracted by the owner after the `lockedUntil` timestamp via `drain()`. The contract stores `referenceToExitRate` mappings for cross-currency base price conversion, used by `CoinvestedPosition` to calculate carry when the exit currency differs from the position's base currency. The contract is funded at initialization via `safeTransferFrom` from a designated currency provider, extends `PayoutBase`, and uses Ownable2Step, ReentrancyGuard, and ERC2771 meta-transactions.

`PriceLinear` is a dynamic pricing oracle implementing a linear function — either rising or falling — based on elapsed time or block count, used by `Crowdinvesting` as an `IPriceDynamic` oracle to adjust the token price over time. It computes the current price by applying a rational slope

(`slopeEnumerator`/`slopeDenominator`) to a base price, with an optional step function via `stepDuration` that produces discrete price changes rather than a continuous line. The owner can update all parameters (slope, start time or block number, step duration, direction) via `updateParameters()`, which triggers a 1-hour cooldown during which `getPrice()` reverts. The contract is deployed via clone/proxy pattern and uses Ownable2Step and ERC2771 meta-transactions.

`TokenSwapBase` is the abstract base contract for token swap variants (`TokenSwap` and `CoinvestedPosition`), providing shared state for token price, currency, token, and receiver, along with initialization logic, fee handling via `IFeeSettingsV3` with an `IFeeSettingsV2` fallback, and pause controls. The owner can set the token price via `setTokenPrice()` at any time without requiring a pause or cooldown. Fee calculation uses `FeeTypes.SECONDARY_MARKET` when the fee settings support V3, and falls back to `privateOfferFee()` for V2-only deployments. The currency must be a `TRUSTED_CURRENCY` on the token's `AllowList`. The contract uses Ownable2StepUpgradeable, PausableUpgradeable, ReentrancyGuardUpgradeable, and ERC2771 meta-transactions.

`PayoutBase` is the abstract base contract for payout variants (`Distribution`, `Exit`), providing shared state for `token`, `currency`, `pricePerToken`, and `lockedUntil`, along with initialization logic, fee handling via `IFeeSettingsV3` (with a silent zero-fee fallback when V3 is not supported), and a `drain()` recovery function. The owner can drain any ERC-20 token held by the contract via `drain()` once `lockedUntil` has elapsed. Fee calculation delegates to `IFeeSettingsV3.fee()` using a caller-specified fee type; if V3 is not supported the fee defaults to zero. Child contracts must implement `eligible()` and `claim()`. The contract uses `Ownable2StepUpgradeable`, `ReentrancyGuardUpgradeable`, and `ERC2771ContextUpgradeable` meta-transactions.

`TimeLock` is a time-locked ERC20 token holder that blocks all withdrawals until a specified `lockedUntil` timestamp, which must be in the future at initialization. The owner can claim dividends from a `Distribution` contract via `claimDistribution()` and exit proceeds from an `Exit` contract (via `GlobalTokenExitRegistry`) via `claimExit()` before the lock expires, forwarding the received currency to a specified recipient. After the lock expires, the owner can `drain()` the full balance of any ERC20 token to a recipient. There is no pause mechanism — the lock is purely time-based. The contract is deployed via clone/proxy pattern and uses Ownable2StepUpgradeable, ReentrancyGuardUpgradeable, and ERC2771 meta-transactions.

`PrivateOffer` is a one-time, single-buyer token purchase contract where the entire deal executes atomically in the constructor: parameters are validated, the platform fee is collected via `IFeeSettingsV2.privateOfferFee()`, the currency is transferred from the buyer's payer address to the company's receiver, and tokens are either minted to the buyer or transferred from a designated `tokenHolder`. The contract is deployed via CREATE2 for deterministic addressing, enabling privacy-preserving flows where allowances can be pre-approved to the computed address before deployment. After construction, the contract holds no state and serves no further purpose. An expiration timestamp is enforced at deployment, and the payment currency must be a `TRUSTED_CURRENCY` on the token's `AllowList`. There are no privileged roles.

`AllowList` is a bitmask-based attribute registry where each address is mapped to a uint256 value encoding attested attributes such as KYC status, citizenship, and tier levels, with 252 attribute bits, 4 cumulative tier bits, and the highest bit (bit 255) reserved for the `TRUSTED_CURRENCY` flag. The owner can set or update attributes for individual addresses or in batch via `set()`, and remove addresses (resetting their attributes to zero) via `remove()`. The contract is used by `Token` to enforce transfer compliance

requirements and by all sale contracts, to verify trusted currencies. It is deployed via clone/proxy pattern, uses Ownable2Step and ERC2771 meta-transactions, and a single `AllowList` instance can serve an unlimited number of `Token` contracts.

`TokenSwap` is a two-way token swap contract supporting both buy and sell orders at a fixed price, extending `TokenSwapBase`. It uses an external `holder` address for liquidity: buyers pay currency to the receiver and receive tokens transferred from the `holder`, while sellers send tokens to the receiver and receive currency transferred from the `holder`. The order size is implicitly capped by the `holder`'s allowance and balance. Fees are collected via `FeeTypes.SECONDARY_MARKET` on both `buy()` and `sell()` operations. `buy()` rounds the currency amount up (ceil) while `sell()` rounds it down (floor). The contract is deployed via clone/proxy pattern and inherits Ownable2StepUpgradeable, PausableUpgradeable, ReentrancyGuardUpgradeable, and ERC2771 meta-transactions from `TokenSwapBase`.

`GlobalTokenExitRegistry` is a singleton registry that stores the authorized `Exit` contract for each `Token`. Once an exit is set for a token via `setExit()`, the binding is immutable and cannot be changed or removed. Authorization is derived from the target token's access control: the caller must hold `DEFAULT_ADMIN_ROLE` on the token or be its `owner()`, with both models checked via try/catch to accommodate different access control implementations. The registry is used by `CoinvestedPosition` and `TimeLock` to locate the exit contract for a given token. It is not upgradeable, supports ERC2771 meta-transactions, and has no admin roles of its own.

Privileged roles

`Crowdinvesting.sol`

- Owner (Ownable2Step):
  - `activateDynamicPricing()` — enables a price oracle to influence the token price (whenPaused).
  - `deactivateDynamicPricing()` — deactivates dynamic pricing, reverting to base price (whenPaused).
  - `setCurrencyReceiver()` — changes the address that receives currency payments (whenPaused).
  - `setMinAmountPerReceiver()` — changes the minimum token amount per receiver (whenPaused).
  - `setMaxAmountPerReceiver()` — changes the maximum token amount per receiver (whenPaused).
  - `setCurrencyAndTokenPrice()` — changes the payment currency and token price; deactivates dynamic pricing (whenPaused).
  - `setMaxAmountOfTokenToBeSold()` — changes the total token supply cap for the sale (whenPaused).
  - `setLastBuyDate()` — sets the auto-pause date after which no purchases are accepted (whenPaused).
  - `setTokenHolder()` — changes the address from which tokens are transferred (whenPaused).
  - `pause()` — pauses the contract, preventing all purchases.
  - `unpause()` — unpauses the contract (requires cooldown to have elapsed).

`FeeSettings.sol`

- Owner (Ownable2Step):
  - `addManager()` — grants manager status to an address.
  - `removeManager()` — revokes manager status from an address.

- `registerFeeType()` — registers a new fee type with hard cap, default rate, and collector.
- `planFeeChange()` — proposes a new default fee numerator for a fee type (12-week delay for increases).
- `executeFeeChange()` — executes a previously planned default fee change after activation date.
- `setDefaultFeeCollector()` — sets the default fee collector address for a fee type.

- Manager (custom `onlyManager` modifier; owner is automatically a manager at initialization):
  - `setCustomFee()` — sets a per-token custom fee discount with a validity date.
  - `removeCustomFee()` — removes a per-token custom fee discount, reverting to default.
  - `setCustomFeeCollector()` — sets a per-token fee collector override for a fee type.
  - `removeCustomFeeCollector()` — removes a per-token fee collector override, reverting to default.

`Vesting.sol`

- Owner (Ownable2StepUpgradeable; owner is automatically a manager at initialization):
  - `addManager()` — grants manager status to an address.
  - `removeManager()` — revokes manager status from an address.
  - `changeBeneficiary()` — changes the beneficiary of a vesting plan (only 1 year after vesting end).
- Manager (custom `onlyManager` modifier):
  - `commit()` — commits to a private vesting plan by storing its hash.
  - `revoke()` — revokes a commitment by setting a new latest end date.
  - `createVesting()` — creates a public vesting plan with allocation, beneficiary, cliff, and duration.
  - `stopVesting()` — stops (early terminates) an active vesting plan.
  - `pauseVesting()` — pauses a vesting plan by stopping it and creating a new one with adjusted terms.
- Beneficiary (per vesting plan):
  - `release()` — releases vested tokens to the beneficiary.
  - `changeBeneficiary()` — changes the beneficiary to a new address.

`Token.sol`

- `DEFAULT_ADMIN_ROLE`:
  - `_authorizeUpgrade()` — authorizes a UUPS proxy upgrade to a new implementation.
  - `setAllowList()` — changes the `AllowList` contract used for compliance enforcement.
  - `acceptNewFeeSettings()` — accepts a previously suggested new `FeeSettings` contract.
  - All role management — can grant/revoke all roles except `TRANSFERER_ROLE` (managed by `TRANSFERERADMIN_ROLE`).
- `REQUIREMENT_ROLE`:
  - `setRequirements()` — changes the transfer compliance requirements bitmask.
- `MINTALLOWER_ROLE`:
  - `increaseMintingAllowance()` — increases the minting allowance for a specific address.
  - `decreaseMintingAllowance()` — decreases the minting allowance for a specific address.
  - `mint()` — can mint unlimited tokens (bypasses minting allowance check).
- `BURNER_ROLE`:
  - `burn()` — burns tokens from any address.
- `TRANSFERERADMIN_ROLE`:

    - Role admin for `TRANSFERER_ROLE` — can grant/revoke `TRANSFERER_ROLE` to/from any address.
- `TRANSFERER_ROLE`:
    - Holders bypass all compliance requirement checks when sending or receiving tokens.
- `PAUSER_ROLE`:
    - `pause()` — pauses the token; no transfers, minting, or burning possible.
    - `unpause()` — unpauses the token.
- `SNAPSHOTCREATOR_ROLE`:
    - `createSnapshot()` — creates a snapshot of all current token balances.
- `FeeSettings` Owner (external):
    - `suggestNewFeeSettings()` — suggests a new `FeeSettings` contract (requires `DEFAULT_ADMIN_ROLE` acceptance).

`CoinvestedPosition.sol`

- Owner (Ownable2StepUpgradeable, inherited from `TokenSwapBase`):
    - `unpause()` — unpauses the contract (requires `tokenPrice` != 0 and `lockedUntil` elapsed).
    - `setCurrency()` — changes the payment currency, base price, and token price(requires `lockedUntil` elapsed).
    - `claimDistribution()` — claims dividends from a `Distribution` contract and splits proceeds.
    - `claimExit()` — claims exit proceeds and splits between receiver and lead investors.
    - `setTokenPrice()` — sets the token sale price (inherited from `TokenSwapBase`).
    - `pause()` — pauses the contract (inherited from `TokenSwapBase`).
    - `withdrawAsCoinvestor()` — withdraw the coinvestor's accumulated credit in specific `_currency` to specified `_receiver`.
    - `ownerRotateLeadInvestorAccount()` - rotate the address of a lead-investor slot.
- Lead Investor (custom role defined during contract initialziation)
    - `rotateLeadInvestorAccount()` - rotate the address that controls a given lead-investor slot.
    - `withdrawAsLeadInvestor()` - withdraw a lead investor's accumulated credit in `_currency` to `_receiver`.

`Distribution.sol`

- Owner (Ownable2Step, inherited from `PayoutBase`):
    - `reassign()` — reassigns unclaimed distribution funds from one address to another (after `lockedUntil`).
    - `drain()` — transfers the entire balance of any ERC20 held by this contract to a recipient (after `lockedUntil`).

`Exit.sol`

- Owner (Ownable2Step, inherited from `PayoutBase`):
    - `drain()` — transfers the entire balance of any ERC20 held by this contract to a recipient (after `lockedUntil`).

`PriceLinear.sol`

- Owner (Ownable2Step):
  - `updateParameters()` — updates the linear price function parameters (slope, start, step duration, direction).

`TokenSwapBase.sol` (abstract)

- Owner (Ownable2StepUpgradeable):
  - `setTokenPrice()` — sets the token sale/buy price.
  - `pause()` — pauses the contract.
  - `unpause()` — abstract, implemented by children.

`PayoutBase.sol` (abstract)

- Owner (Ownable2Step):
  - `drain()` — transfers the full balance of any ERC-20 token held by the contract to a chosen recipient, only after `lockedUntil` has passed.

`TimeLock.sol`

- Owner (Ownable2StepUpgradeable):
  - `claimDistribution()` — claims dividends from a `Distribution` contract and forwards to a recipient.
  - `claimExit()` — claims exit proceeds for locked tokens and forwards to a recipient.
  - `drain()` — transfers the full balance of any ERC20 to a recipient (only after `lockedUntil`).

`PrivateOffer.sol`

No privileged roles. All logic executes atomically in the constructor. No persistent state or admin functions exist after deployment.

`AllowList.sol`

- Owner (Ownable2Step):
  - `set(address, uint256)` — sets or updates the attribute bitmask for a single address.
  - `set(address[], uint256[])` — batch sets attributes for multiple addresses.
  - `remove(address)` — removes a single address from the allow list.
  - `remove(address[])` — batch removes multiple addresses from the allow list.

`TokenSwap.sol`

- Owner (Ownable2StepUpgradeable, inherited from `TokenSwapBase`):
  - `unpause()` — unpauses the contract.
  - `setTokenPrice()` — sets the token price (inherited from `TokenSwapBase`).
  - `pause()` — pauses the contract (inherited from `TokenSwapBase`).

`GlobalTokenExitRegistry.sol`

- `Token` `DEFAULT_ADMIN_ROLE` Holder or `Token` Owner (authorization derived from the target `Token` contract, not from the registry itself):
  - `setExit()` — registers an `Exit` contract for a `Token` (one-time, immutable binding). Caller must hold `DEFAULT_ADMIN_ROLE` on the token or be the token's `owner()`.

Potential Risks

- **Centralized Control Over Transfer Restrictions and Allowlist Logic:** The token contract enforces requirement checks on every transfer, relying on user attributes stored in an external allowlist contract. The contract owner can arbitrarily modify user attributes, while privileged roles can update both the transfer requirements and the allowlist contract address. This design introduces a centralization risk, where changes to requirements, user attributes, or the allowlist implementation could block transfers for specific users or the entire system. Additionally, if DeFi protocols are not properly designated as privileged, user interactions such as fund recovery or token swaps may be restricted, potentially leading to loss of access to funds.
- **Owner-Controlled Base Price in CoinvestedPosition.claimExit Enables Carry Manipulation:** When the exit currency differs from the stored investment `currency` and no `referenceToExitRate` is configured, `claimExit()` falls back to an owner-supplied `_basePrice` parameter with no on-chain validation against market rates. The owner can exploit this by inflating `_basePrice` so that `basePayout` equals or exceeds the total received amount, effectively zeroing the carry owed to lead investors. This creates a trust dependency where a malicious or compromised owner can silently steal carry from lead investors without any safeguard or independent price verification.
- **GlobalTokenExitRegistry Exit Assignment Is Permanent and Irreversible:** The `setExit()` function in `GlobalTokenExitRegistry` uses an `ExitAlreadySet` check that prevents any update once an exit contract is assigned to a token. There is no emergency override, governance mechanism, or admin function to reassign the exit contract. If the assigned exit contract contains a bug, becomes compromised, or needs to be upgraded, the registry offers no recovery path, permanently locking the token to the faulty exit contract.
- **FeeSettings Manager Can Redirect Fee Collection Without Owner Approval:** The `setCustomFeeCollector()` function in `FeeSettings` is gated by `onlyManager` rather than `onlyOwner`, and has no timelock or owner approval requirement. A compromised or malicious manager can redirect all platform fee revenue for any token to an attacker-controlled address. This represents a significant trust assumption, as the manager role alone has full authority over fee collection destinations without additional oversight.
- **Scope Definition and Security Guarantees**: The audit does not cover all code in the repository. Contracts outside the audit scope may introduce vulnerabilities, potentially impacting the overall security due to the interconnected nature of smart contracts.
- **Absence of Time-lock Mechanisms for Critical Operations**: Without time-locks on critical operations, there is no buffer to review or revert potentially harmful actions, increasing the risk of rapid exploitation and irreversible changes.
- **Single Points of Failure and Control**: The project is fully or partially centralized, introducing single points of failure and control. This centralization can lead to vulnerabilities in decision-making and operational processes, making the system more susceptible to targeted attacks or manipulation.
- **Owner's Unrestricted State Modification**: The absence of restrictions on state variable modifications by the owner leads to arbitrary changes, affecting contract integrity and user trust, especially during critical operations like minting phases.
- **Administrative Key Control Risks**: The digital contract architecture relies on administrative keys for critical operations. Centralized control over these keys presents a significant security risk, as compromise or misuse can lead to unauthorized actions or loss of funds.
- **Owner-Controlled Price Setting Across Contracts**: The token price used for buying, selling, and exit settlements is directly provided by the owner without independent price discovery or external

verification. In `TokenSwapBase.setTokenPrice()`, the owner may change the price at any time without pause or cooldown constraints. In `Crowdinvesting`, the owner selects the oracle address and sets both the base price and the min/max bounds that constrain oracle output. In `CoinvestedPosition.claimExit()`, the owner supplies a fallback base price parameter that determines the carry/receiver split when no on-chain rate exists.

- **Trusted Forwarder Can Impersonate Any Address Across All Platform Contracts:** All core contracts inherit `ERC2771ContextUpgradeable` with an immutable trusted forwarder set at construction. This forwarder can submit transactions impersonating any address, enabling it to execute transfers, approvals, minting, and role-gated functions on behalf of any user. Since the forwarder cannot be rotated or revoked post-deployment, a compromised forwarder represents a single point of failure capable of draining tokens and bypassing access control protocol-wide.
- **Allowlist Requirements Can Freeze User Tokens Without Recourse:** The Token contract enforces a bitmask check (`allowList.map(_address) & requirements == requirements`) on every transfer for both sender and receiver. If a user's AllowList attributes are revoked, requirements are raised after distribution, or the AllowList contract is replaced, all transfers for affected users are blocked immediately. There is no self-service appeal, cooldown, or governance mechanism — token recovery depends entirely on privileged actors restoring attributes or granting the `TRANSFERER_ROLE`.
- **Unrestricted Burn From Any Address by `BURNER_ROLE`**: The `BURNER_ROLE` in the `Token` contract can burn tokens from any holder without their consent. While this appears intended for legal recovery and lost-key reissuance flows, the current NatSpec only describes the mechanical burn behavior and does not document the intended use cases, operational preconditions, or governance process required before execution. If the BURNER_ROLE is compromised or maliciously assigned, an attacker could arbitrarily destroy user balances.
- **Owner Recovery Mechanism Requires Lead Investor Vigilance:** The 90-day owner recovery mechanism is documented and appears to be an intentional trust trade-off. However, lead investors must actively monitor the `recoveryArmedAt` state and disarm recovery, either through withdrawal or self-rotation, to prevent the owner from completing a takeover after the recovery delay. Since the owner can repeatedly trigger small credit events, such as selling 1 token, the recovery timer may be re-armed over time, which may lead to unintended owner recovery execution if lead investors are not actively monitoring the recovery state.

Findings

Vulnerability Details

F-2026-16556 - Push Based Settlement With Immutable Recipients Enables Permanent Denial Of Service - Medium

Description:

The `CoinvestedPosition._settle()` function distributes profit by iterating over the `leadInvestors` array and performing `_currency.safeTransfer()` operations to each recipient within the same transaction.

```
function _settle(uint256 profit, IERC20 _currency) internal {
    require(address(_currency) != address(token), CurrencyEqualsToken());
    for (uint256 i = 0; i < leadInvestors.length; i++) {
        uint256 carryFraction = (uint256(leadInvestors[i].profitFraction) * p
rofit) / type(uint64).max;
        if (carryFraction != 0) {
            _currency.safeTransfer(leadInvestors[i].account, carryFraction);
        }
    }
    _currency.safeTransfer(receiver, _currency.balanceOf(address(this)));
}
```

The implementation uses a push-based transfer model without any failure handling. No try/catch mechanism, skip-on-failure logic, or deferred withdrawal pattern is implemented.

If any recipient address causes `_currency.safeTransfer()` to revert, the entire transaction reverts. This condition is particularly relevant for blacklist-enabled tokens, where transfers to restricted addresses are disallowed.

The `leadInvestors` array is immutable after initialization, and no mechanism exists to modify, remove, or replace entries. As a result, if any recipient becomes non-transferable, all settlement-related execution paths revert.

The `CoinvestedPosition._settle()` function is invoked by `CoinvestedPosition.buy()`, `CoinvestedPosition.claimDistribution()`, and `CoinvestedPosition.claimExit()`. A single failing transfer causes all these operations to become permanently unavailable. This results in a complete denial of service, preventing distribution, exit, and other settlement-related actions, while assets remain locked within the contract.

Assets:

- ./CoinvestedPosition.sol [https://github.com/corpus-io/tokenize.it-smart-contracts.git]

Status: Fixed

Classification

Impact Rate: 5/5

Likelihood Rate: 3/5

Exploitability: Semi-Dependent

Complexity: Simple

Severity: Medium

Recommendations

Remediation: It is recommended to replace the push-based settlement mechanism with a pull-based pattern, where accrued balances are stored on-contract and recipients withdraw their allocations independently. This prevents a single failing transfer from blocking the entire settlement process and removes dependency on recipient transferability within critical execution paths.

Resolution: Fixed in `d5e136b`. The `CoinvestedPosition` contract was refactored from a push-based settlement model to a pull-based credit/withdrawal pattern. The `_settle()` function, which performed direct `safeTransfer()` calls to each lead investor and the receiver within a single transaction, was replaced by `_credit()`, which records accrued balances in on-contract `leadInvestorCredit` and `coinvestorCredit` mappings keyed by currency. Recipients independently withdraw their allocations via `withdrawAsLeadInvestor()` and `withdrawAsCoinvestor()`, each protected by `nonReentrant`. This eliminates the single-point-of-failure where a blacklisted or reverting recipient could permanently block all settlement operations (`buy()`, `claimDistribution()`, `claimExit()`). Additionally, the `receiver` field was removed from `TokenSwapBase` and the coinvestor role was merged with the contract owner, who specifies a destination address at withdrawal time. A lead investor account rotation mechanism (`rotateLeadInvestorAccount()`) and an owner-driven recovery path with a 90-day timeout (`ownerRotateLeadInvestorAccount()`) were introduced to handle key loss scenarios.

F-2026-16595 - Inconsistent Validation of LockedUntil Enables Premature Privileged Actions - Medium

Description:

A `lockedUntil` timestamp is persisted across multiple contracts to restrict the execution of privileged functions until a specified future time. However, validation of this value during initialization is inconsistent and, in some cases, entirely absent. Validation at initialization is inconsistent:

- `Exit.initialize()` enforces `_arguments.lockedUntil > block.timestamp`.
- `TimeLock.initialize()` enforces `_lockedUntil > block.timestamp`, but its own NatSpec documents `"0 means no lock"`, contradicting the check.
- `Distribution.initialize()` forwards `_arguments.lockedUntil` to `__PayoutBase_init` with no validation. A zero or past value permits the owner to call `Distribution.reassign()` and `PayoutBase.drain()` immediately.
- `CoinvestedPosition.initialize()` accepts any `_arguments.lockedUntil` with no validation. A zero or past value allows `CoinvestedPosition.unpause()` and `CoinvestedPosition.setCurrency()` to be called immediately, defeating the advertised lock window; the struct NatSpec further states "0 means no lock", which again contradicts the validation applied elsewhere.

Due to these inconsistencies, time-based restrictions can be bypassed. Assets intended for distribution may be drained before claims are executed, and positions expected to remain locked can be modified or sold without delay. This behavior contradicts the documented invariants and undermines the intended security guarantees of the system.

Assets:

- ./CoinvestedPosition.sol [https://github.com/corpus-io/tokenize.it-smart-contracts.git]
- ./Distribution.sol [https://github.com/corpus-io/tokenize.it-smart-contracts.git]
- ./Exit.sol [https://github.com/corpus-io/tokenize.it-smart-contracts.git]
- ./TimeLock.sol [https://github.com/corpus-io/tokenize.it-smart-contracts.git]

Status: Fixed

Classification

Impact Rate: 3/5

Likelihood Rate: 3/5

Exploitability: Independent

Complexity: Simple

Severity: Medium

Recommendations

Remediation: It is recommended to apply the following changes uniformly across all four contracts:

Enforce a strictly future `lockedUntil` at initialization in every contract that declares the variable. Centralize the check in `PayoutBase.__PayoutBase_init()` so `Distribution` (and any future `PayoutBase` subclass) inherits it, and add the same check to `CoinvestedPosition.initialize()`:

```
require(_lockedUntil > block.timestamp, LockedUntilNotInFuture());
```

Enforce a minimum lock duration (e.g., 30 days) so the lock period cannot be set to a trivially short window that bypasses the protective intent:

```
uint256 constant MIN_LOCK_DURATION = 30 days;
require(_lockedUntil >= block.timestamp + MIN_LOCK_DURATION, LockDurationTooS
hort());
```

The constant should be defined once (e.g., alongside the existing errors in `common/Errors.sol` or in `PayoutBase`) and referenced by all consumers to keep the minimum consistent.

Align NatSpec with the implementation so no contradiction remains. In particular:

- `TimeLock.sol` line 52: remove the `"0 means no lock"` wording and document that `_lockedUntil` must be at least `block.timestamp + MIN_LOCK_DURATION`.
- `CoinvestedPosition.sol` line 33 (the `lockedUntil` field on `CoinvestedPositionInitializerArguments`): remove the `"0 means no lock"` wording and reflect the new minimum-duration requirement.
- Any further NatSpec that refers to a zero-value bypass should be revised to match.

Applying these three changes together removes the bypass paths in `Distribution` and `CoinvestedPosition`, prevents trivially short lock windows in `Exit` and `TimeLock`, and eliminates the documentation-vs-code mismatch that currently misleads integrators.

Resolution: Fixed in `9125291`: The `lockedUntil` validation was was clarified and made consistent with the intended locking semantics across the affected contracts.

In `PayoutBase.__PayoutBase_init()`, a `require(_lockedUntil >= block.timestamp + 30 days, LockedUntilNotInFuture())` check was added, enforcing a minimum 30-day lock duration for both `Distribution` and `Exit` (which inherit from `PayoutBase`). The redundant `require(_arguments.lockedUntil > block.timestamp)` check was removed from `Exit.initialize()` since the stricter base class check supersedes it. In `TimeLock.initialize()`, the check was strengthened from `require(_lockedUntil > block.timestamp)` to `require(_lockedUntil >= block.timestamp + 30 days)`.

`CoinvestedPosition` remains intentionally exempt from the 30-day minimum timelock requirement. Its primary purpose is to facilitate the split of proceeds between the coinvestor and lead investors when proceeds are received, which may occur shortly after the investment in some cases. The timelock is optional for `CoinvestedPosition` and is only relevant when the company being invested in requires a guaranteed holding period. Since this is not always required, the contract deliberately accepts `lockedUntil` values that may already have expired. However, `0` is still rejected to prevent an omitted or forgotten field from silently degrading into a "no lock" configuration: `require(_arguments.lockedUntil > 0, ZeroLockedUntil());`

The NatSpec was updated to reflect the new validation semantics, removing the contradictory `"0 means no lock"` wording from `TimeLock` and `CoinvestedPosition`. The error description for `LockedUntilNotInFuture` was updated from `"not strictly in the future"` to `"does not meet the minimum lock duration.`

F-2026-16480 - Missing Base Price Validation In Initialization Enables Full Profit Redirection - Low

Description:

The `CoinvestedPosition.setCurrency()` function enforces `_basePrice > 0`, while the `CoinvestedPosition.initialize()` function accepts `_arguments.basePrice` without validation, allowing it to be set to zero.

```
function setCurrency(IERC20 _currency, uint256 _basePrice) external onlyOwner
{
   require(block.timestamp >= lockedUntil, TimeLockNotExpired());
   require(address(_currency) != address(0), ZeroCurrencyAddress());
   require(address(_currency) != address(token), CurrencyEqualsToken());
   require(_basePrice > 0, ZeroPrice());
   require(token.allowList().map(address(_currency)) == TRUSTED_CURRENCY, Unt
rustedCurrency());
   basePrice = _basePrice;
   currency = _currency;
}

function initialize(CoinvestedPositionInitializerArguments memory _arguments)
external initializer {
    _initializeBase(_arguments.owner, 0, _arguments.baseCurrency, _arguments.
token,
                    _arguments.receiver);

    require(_arguments.leadInvestors.length > 0, NoLeadInvestors());
    uint64 profitFractionsSum = 0;
    for (uint256 i = 0; i < _arguments.leadInvestors.length; i++) {
        require(_arguments.leadInvestors[i].account != address(0), ZeroLeadIn
vestorAddress());
        require(_arguments.leadInvestors[i].profitFraction > 0, ZeroLeadInves
torProfitFraction());
        profitFractionsSum +=
            _arguments.leadInvestors[i]
                .profitFraction;  // reverts on overflow, thus avoiding profi
tFractionsSum > 100%
        leadInvestors.push(_arguments.leadInvestors[i]);
    }
    require(address(_arguments.tokenExitRegistry) != address(0), ZeroTokenExi
tRegistryAddress());
    basePrice = _arguments.basePrice;
    lockedUntil = _arguments.lockedUntil;
    tokenExitRegistry = _arguments.tokenExitRegistry;

    // Pausing the contract prevents an immediate sell of the tokens. Once th
```

```
ey should be sold,
    // update price and unpause.
    _pause();
}
```

When `basePrice = 0`, the payout logic in `CoinvestedPosition.buy()` results in `payoutCoinvestor = 0`, causing the entire remaining amount to be treated as profit. Consequently, the full proceeds are distributed according to `profitFraction` among lead investors, while the coinvestor receives only negligible rounding residuals.

A similar effect occurs in `CoinvestedPosition.claimExit()`, where `basePayout = 0` results in all proceeds being classified as profit and distributed accordingly.

The intended design specifies that the coinvestor receives `basePrice * tokenAmount` prior to profit distribution. Setting `basePrice` to zero violates this invariant and removes the guaranteed base allocation, resulting in a complete redirection of proceeds away from the coinvestor.

Assets:

- ./CoinvestedPosition.sol [https://github.com/corpus-io/tokenize.it-smart-contracts.git]

Status: Fixed

Classification

Impact Rate: 2/5

Likelihood Rate: 3/5

Exploitability: Dependent

Complexity: Simple

Severity: Low

Recommendations

Remediation: It is recommended to enforce a non-zero base price during initialization by adding a validation check ensuring `_arguments.basePrice > 0`, maintaining consistency with the invariant enforced in `CoinvestedPosition.setCurrency()`.

Resolution: The issue was fixed in commit `ea9bc2d`. A `require(_arguments.basePrice > 0, ZeroPrice())` check was added to `CoinvestedPosition.initialize()`, preventing initialization with a zero base price that would redirect all proceeds as profit to lead investors.

F-2026-16481 - Currency Update Without Token Price Sync Leads To Mispricing - Low

Description:

The `CoinvestedPosition.setCurrency()` function updates the currency and adjusts `basePrice` to the new currency's units but does not update `tokenPrice`, which is used by `CoinvestedPosition.buy()`. As a result, `tokenPrice` may remain denominated in the previous currency configuration.

```
function setCurrency(IERC20 _currency, uint256 _basePrice) external onlyOwner
{
    require(block.timestamp >= lockedUntil, TimeLockNotExpired());
    require(address(_currency) != address(0), ZeroCurrencyAddress());
    require(address(_currency) != address(token), CurrencyEqualsToken());
    require(_basePrice > 0, ZeroPrice());
    require(token.allowList().map(address(_currency)) == TRUSTED_CURRENCY, Un
trustedCurrency());
    basePrice = _basePrice;
    currency = _currency;
}
```

The function does not enforce a paused state before execution, and no automatic pause is triggered during the currency update. Additionally, `CoinvestedPosition.unpause()` validates only that `tokenPrice != 0`, allowing the contract to remain active while `tokenPrice` is inconsistent with the updated currency.

When the currency is changed to an asset with different decimals or valuation, `CoinvestedPosition.buy()` calculates the required payment using the outdated `tokenPrice`:

```
currencyAmount = ceilDiv(_tokenAmount * tokenPrice, 10 ** token.decimals())
```

This results in incorrect pricing denominated in the new currency. For example, transitioning from a 6-decimal currency to an 18-decimal currency without adjusting `tokenPrice` leads to a severe underpricing condition, where assets can be acquired for a near-zero amount.

Additionally, a race condition exists between sequential administrative operations. After `CoinvestedPosition.setCurrency()` is executed and before a corresponding `TokenSwapBase.setTokenPrice()` update, transactions may be executed using the stale pricing configuration, enabling exploitation during this intermediate state.

Assets:

- ./CoinvestedPosition.sol [https://github.com/corpus-io/tokenize.it-smart-contracts.git]

Status: Fixed

Classification

Impact Rate: 4/5

Likelihood Rate: 3/5

Exploitability: Dependent

Complexity: Simple

Severity: Low

Recommendations

Remediation: It is recommended to enforce synchronization between currency and token price by requiring `tokenPrice` to be updated atomically with the currency change

Resolution: The issue was fixed in commit `ae7896a`. The `CoinvestedPosition.setCurrency()` function now requires a `_tokenPrice` parameter and sets `tokenPrice` atomically together with `currency` and `basePrice`, eliminating the race condition and decimal-mismatch mispricing window.

F-2026-16558 - Privileged Burn Function Allows Arbitrary Token Destruction - Low

Description:

The `Token.burn()` function permits any address holding `BURNER_ROLE` to destroy an arbitrary amount of tokens from any holder's balance without holder consent, allowance, or prior notification:

```
/**
* @notice Burn `_amount` tokens from `_from`.
* @param _from address that holds the tokens
* @param _amount how many tokens to burn
*/
function burn(address _from, uint256 _amount) external onlyRole(BURNER_ROLE)
{
    _burn(_from, _amount);
}
```

This capability is intentional and serves a legitimate purpose: it is the on-chain primitive used to satisfy legal obligations and to recover tokens in operational error scenarios (for example, the established `Token.burn()` + `Token.mint()` pattern that reissues tokens to a new address when a holder loses access to their key). The role-level NatSpec acknowledges this:

```
// Token.sol Lines 39-40
/// @notice The role that has the ability to burn tokens from anywhere. Usage
is planned for legal purposes and error recovery.
bytes32 public constant BURNER_ROLE = keccak256("BURNER_ROLE");
```

The issue is not the existence of the capability but the disconnect between its sensitive, recovery-oriented purpose and the way it is surfaced at the function level:

1. The function NatSpec for `Token.burn()` describes only the mechanical effect ("Burn `_amount` tokens from `_from`") and does not state the intended use cases (legal recovery, lost-key reissuance via burn-then-mint), the operational preconditions expected of the admin, or the expectation that calls should be infrequent and tied to documented off-chain procedures.
2. The function name `burn` is generic and indistinguishable from a standard ERC-20 burn-from-self pattern. Block explorers, integrators, and downstream tooling reading the ABI cannot distinguish this privileged "burn-from-anyone" recovery primitive from an ordinary user-initiated burn.

3. There is no on-chain signal that the action is extraordinary — no dedicated event beyond the standard `Transfer(_, address(0), _)`, no descriptive reason parameter, and no naming convention that flags the call as a recovery action when it appears in a transaction history.
4. Project-level documentation references the recovery use case in `docs/dividends_and_exit.md` ("Recovery mechanism to help users: mint & burn") and in `docs/upgradeability.md` (admin "can burn anyones token at any time"), but the function-level documentation surfaced through the contract source and ABI does not reflect this framing.

The combined effect is reduced transparency for token holders and third-party observers, who must reconstruct the intended semantics of `Token.burn()` from scattered project documentation rather than from the contract's own NatSpec. In the event of admin key compromise or misuse, the lack of on-chain framing also makes it harder to distinguish a legitimate recovery action from an abuse, complicating after-the-fact accountability, which is the central mitigation cited for arbitrary burn authority in `docs/upgradeability.md`.

Assets:

- ./Token.sol [https://github.com/corpus-io/tokenize.it-smart-contracts.git]

Status: Accepted

Classification

Impact Rate: 4/5

Likelihood Rate: 2/5

Exploitability: Semi-Dependent

Complexity: Simple

Severity: Low

Recommendations

Remediation: It is recommended to align the on-chain surface with the function's documented recovery-only intent. Specifically:

- Expand the `Token.burn()` NatSpec to explicitly state the intended use cases: legal compliance actions, error recovery, and the burn-then-mint pattern for reissuing tokens from a lost address. The NatSpec should also note that the function is expected to be invoked rarely and only in connection with off-chain procedures defined by the company.

- Consider renaming the function to a name that reflects its privileged nature, such as `Token.recoveryBurn()`, `Token.adminBurn()`, or `Token.emergencyBurn()`. A dedicated name distinguishes this primitive from a generic ERC-20 burn in block explorers, indexers, and integrator code, making misuse more visible.
- Consider emitting a dedicated event in addition to the standard `Transfer` event — for example, `RecoveryBurn(address indexed from, uint256 amount, address indexed operator)` — so that recovery actions are unambiguously distinguishable in event logs without parsing transaction context.
- Add or expand a section in `docs/dev_overview.md` (or a dedicated document) that describes the recovery procedure end-to-end: when `Token.burn()` should be used, the expected pairing with `Token.mint()` for lost-key reissuance, the off-chain authorization process, and the transparency mechanisms that hold the admin accountable.

Resolution: The issue was accepted by the Tokenize.it team with the following statement:

Design Intent

`Token.burn()` allows any address holding the BURNER_ROLE to remove tokens from any address. This capability is intentional and serves two distinct purposes.

1. **Token recovery.** Investors may lose access to the address holding their tokens — for example, because a private key is no longer accessible. If the tokens have never moved and the investor's identity can be re-verified through the same KYC process used at the time of investment, the company admin can burn the tokens from the inaccessible address and re-mint the same amount to a new address of the investor's choosing. Without this mechanism, a verifiable investor would have no path to reclaim their position.
2. **Legal compliance.** The tokens issued through this framework are legally bound to the company and the investors through off-chain contracts. Courts or regulatory authorities may order the removal of specific tokens. The "law is law" principle applies here: a framework that cannot follow a lawful order is not suitable for regulated equity tokenisation. The burn function provides the means to comply.

Auditability as the Primary Guarantee

The tokenize.it framework prioritises auditability and tradeability over decentralisation. Every burn emits both the standard ERC-20 `Transfer(from, address(0), amount)` event and a dedicated `Burn(address indexed from, uint256 amount, address indexed operator)` event. The dedicated event makes privileged burns unambiguously distinguishable from ordinary transfers in event logs, block explorers, and integrator tooling. Every such record is permanent and publicly visible, so any abuse — or perceived abuse — of

this power can be identified, attributed, and challenged through legal channels. This traceability is a stronger real-world guarantee than a technical impossibility of burning, which would only shift the risk to the upgrade mechanism anyway (see below).

The Burn Function Does Not Introduce New Trust Assumptions

Because the token is deployed behind an ERC1967 upgradeable proxy, the DEFAULT_ADMIN_ROLE already has the power to do anything — including burning tokens — by upgrading the logic contract. The explicit `burn()` function and BURNER_ROLE do not widen the trust surface; they make the capability visible, nameable, and delegable.

This separation matters in practice: routine burn operations (recovery requests, compliance orders) can be assigned to a dedicated address without granting that address the far more powerful admin role, which also controls upgrades, role management, and every other privileged action.

Operational Security Recommendation

Because the DEFAULT_ADMIN_ROLE is the root of all privilege — including the ability to upgrade the contract, revoke roles, and implicitly burn any token — it must be held by a well-configured multisig with a meaningful signing threshold. A compromised admin key is equivalent to a compromised token contract. A multisig raises the bar for an attacker and creates an auditable approval trail for every sensitive action taken by the company.

F-2026-16606 - Lack of Exit Token Validation Allows Mismatched Contract Binding - Low

Description:

The `GlobalTokenExitRegistry.setExit()` function registers an `Exit` contract for a given `Token` after verifying that the caller holds administrative privileges over the token and that no prior `Exit` contract has been registered. However, no validation is performed to ensure that the provided `Exit` contract is associated with the same token. Specifically, the relation `_exit.token() == _token` is not enforced.

```
function setExit(Token _token, Exit _exit) external {
    require(address(_token) != address(0), ZeroTokenAddress());
    require(address(_exit) != address(0), ZeroExitAddress());
    require(address(exits[_token]) == address(0), ExitAlreadySet());
    require(_isTokenAdminOrOwner(_token, _msgSender()), NotTokenAdminOrOwner(
));
    exits[_token] = _exit;
    emit ExitSet(_token, _exit);
}
```

As a result, an `Exit` instance initialized for a different token can be registered under an unrelated token entry. Since the registry mapping is immutable after the initial assignment, any subsequent registration attempt for the same token is reverted. No replacement or override mechanism exists, causing the misregistration to remain permanent for the lifetime of the registry.

This misconfiguration affects the downstream exit-claim pipeline. Both `TimeLock.claimExit()` and `CoinvestedPosition.claimExit()` retrieve the registered `Exit` contract from the registry, approve the `Exit` contract for the balance of the expected token, and invoke `exit.claim()`.

If the registered `Exit` contract is bound to a different token, the internal claim logic operates on an unintended asset. Balance checks are performed against the wrong token, causing execution to revert either due to `NothingToClaim()` or due to insufficient allowance during `safeTransferFrom()`. As a result, the exit-claim path becomes permanently inaccessible for the affected token.

The impact extends differently across dependent contracts. In `TimeLock`, a partial fallback remains available through `TimeLock.drain()` after the expiration of the lock period, allowing recovery of locked assets under specific conditions. In `CoinvestedPosition`, no equivalent fallback mechanism exists for the exit-settlement flow, causing distribution of carry-related assets to remain permanently inaccessible for the affected token.

The issue does not involve direct asset theft, as token allowances are granted and consumed within the same transaction. The impact is limited to denial-of-service and permanent disruption of the exit-settlement mechanism under irreversible misconfiguration.

Assets:

- ./GlobalTokenExitRegistry.sol [https://github.com/corpus-io/tokenize.it-smart-contracts.git]

Status: Fixed

Classification

Impact Rate: 5/5

Likelihood Rate: 2/5

Exploitability: Dependent

Complexity: Simple

Severity: Low

Recommendations

Remediation: A token consistency check should be enforced during `Exit` registration to ensure that the registered `Exit` contract is bound to the intended token. Validation of the token association should be introduced before persisting the registry entry, for example:

```
require(address(_exit.token()) == address(_token), ExitTokenMismatch());
```

This validation ensures consistency between the registry entry and the Exit contract configuration, preventing irreversible misregistration and preserving the availability of the exit-settlement flow.

Resolution: The issue was fixed in commit `9b999b5`. A `require(address(_exit.token()) == address(_token), ExitTokenMismatch())` check was added to `GlobalTokenExitRegistry.setExit()`, preventing registration of an `Exit` contract whose `token()` does not match the token being registered.

F-2026-16644 - Asymmetric Parameter Update Logic Allows Immediate Price Availability - Low

Description:

The `PriceLinear.updateParameters()` function updates pricing parameters through the internal `PriceLinear._updateParameters()` function and subsequently updates `coolDownStart` to the current block timestamp. This establishes a cooldown period during which `PriceLinear.getPrice()` reverts until the configured cooldown duration has elapsed.

```
function updateParameters(...) external onlyOwner {
    _updateParameters(_slopeEnumerator, _slopeDenominator, _startTimeOrBlockN
umber, _stepDuration,
                      _isBlockBased, _isRising);
    coolDownStart = block.timestamp;
}
```

However, this behavior is not mirrored during initialization. The `PriceLinear.initialize()` function invokes `PriceLinear._updateParameters()` directly without updating `coolDownStart`. As a result, the `coolDownStart` storage slot retains its default value of `0` after deployment.

```
function initialize(...) external initializer {
    require(_owner != address(0), ZeroOwnerAddress());
    __Ownable2Step_init();       // sets msgSender() as owner
    _transferOwnership(_owner);  // sets owner as owner
    _updateParameters(_slopeEnumerator, _slopeDenominator, _startTimeOrBlockN
umber, _stepDuration,
                      _isBlockBased, _isRising);
}
```

The cooldown validation is enforced in `PriceLinear.getPrice():`

```
function getPrice(uint256 basePrice) public view returns (uint256) {
    require(coolDownStart + COOL_DOWN_DURATION < block.timestamp, CoolDownNot
Over());
    // ...
}
```

With `coolDownStart` left at zero, this condition resolves to a static comparison against the cooldown duration. Since current block timestamps are significantly greater than the cooldown threshold, the condition is immediately satisfied after initialization.

As a result, the first parameter set established during initialization becomes effective immediately, while all subsequent parameter changes performed through `PriceLinear.updateParameters()` remain subject to the cooldown restriction.

This creates inconsistent behavior between the contract's two parameter write paths and breaks the implicit invariant that every parameter update should be followed by a quarantine period before price retrieval becomes available.

Assets:

- ./PriceLinear.sol [https://github.com/corpus-io/tokenize.it-smart-contracts.git]

Status: Fixed

Classification

Impact Rate: 2/5

Likelihood Rate: 5/5

Exploitability: Dependent

Complexity: Simple

Severity: Low

Recommendations

Remediation: The `coolDownStart` state update should be moved into `PriceLinear._updateParameters()` so that all parameter write paths enforce identical cooldown behavior. This ensures that both initialization and post-deployment parameter updates apply the same quarantine window before `PriceLinear.getPrice()` becomes callable.

If the differing behavior between initialization and post-initialization updates is intentional, the contract documentation should explicitly define this distinction to prevent incorrect assumptions in downstream integrations.

Resolution: The issue was fixed in commit `5ebdcc9`. The `coolDownStart = block.timestamp` assignment was moved from the public `updateParameters()` wrappers into the internal `_updateParameters()` helpers. This ensures the cooldown is consistently triggered during both initialization and post-deployment parameter updates, preventing immediate price availability after deployment.

F-2026-16661 - Max Purchase Limit Is Applied Per Receiver Instead of Per Buyer - Low

Description:

`Crowdinvesting._checkAndDeliver()` function enforces the purchase cap through the condition:

```
tokensBought[_tokenReceiver] + _amount <= maxAmountPerBuyer
```

This accounting model tracks purchased token amounts by the receiver address (`_tokenReceiver`) rather than by the transaction originator (`_msgSender()`).

As a result, the purchase cap is effectively enforced on a per-receiver basis rather than on a per-buyer basis. This allows a single buyer to bypass the intended allocation restriction by distributing purchases across multiple receiver addresses under the same control. Since each receiver address maintains an independent accounting entry, the cumulative purchased amount can exceed `maxAmountPerBuyer` without violating the enforced condition.

The protocol documentation defines the invariant as:

> "No token amount that is larger than maxAmountPerBuyer can be bought by one address through this contract."

While this statement remains technically accurate if interpreted as a receiver-level restriction, it creates ambiguity because the natural interpretation suggests a buyer-level cap. The current implementation does not enforce that stronger interpretation.

This discrepancy weakens the effectiveness of allocation limits and permits concentration of token distribution by a single participant through address fragmentation.

Assets:

- ./Crowdinvesting.sol [https://github.com/corpus-io/tokenize.it-smart-contracts.git]

Status: Fixed

Classification

Impact Rate: 2/5

Likelihood Rate: 3/5

Exploitability: Independent

Complexity: Simple

Severity: Low

Recommendations

Remediation: Purchase cap semantics should be explicitly aligned between implementation and documentation.

Two possible approaches exist:

1. **Clarify Receiver-Based Cap Semantics**
   Documentation should explicitly state that `maxAmountPerBuyer` is enforced per receiver address rather than per purchasing entity.
2. **Enforce Buyer-Level Purchase Accounting**
   Purchase accounting should be extended to track `_msgSender()` in addition to `_tokenReceiver`, ensuring that aggregate purchases initiated by the same buyer remain bounded by `maxAmountPerBuyer`.

The second approach preserves buyer-level allocation guarantees and prevents circumvention through receiver address rotation.

Resolution: The issue was fixed in commit `3393af7`. The `Crowdinvesting` contract's purchase limit tracking was renamed from `tokensBought` / `minAmountPerBuyer` / `maxAmountPerBuyer` to `tokensBoughtByReceiver` / `minAmountPerReceiver` / `maxAmountPerReceiver` with updated NatSpec to explicitly document that limits are enforced on the `_tokenReceiver` address, not on the transaction originator (`_msgSender()`).

F-2026-16745 - Vesting.commit Overwrites Existing Commitment State, Including Revoked or Already-Revealed Hashes - Low

Description: `Vesting.commit()` unconditionally writes `commitments[_hash] = type(uint64).max`, regardless of the prior state of the hash.

```
/**
* Managers can commit to a vesting plan without revealing its details.
* The parameters are hashed and this hash is stored in the commitments mappin
g.
* Anyone can then reveal the vesting plan by providing the parameters and the
salt.
* @param _hash commitment hash
*/
function commit(bytes32 _hash) external onlyManager {
   require(_hash != bytes32(0), ZeroHash());
   // the value is interpreted as maximum end date of the vesting
   // for real world use cases, type(uint64).max is "unlimited"
   commitments[_hash] = type(uint64).max;
   emit Commit(_hash);
}
```

Three abuse paths arise:

1. **Resurrected revocation.** A manager revokes a commitment via `Vesting.revoke()` (setting `commitments[_hash]` to a value < `type(uint64).max`). Subsequently, a manager (potentially the same one) calls `Vesting.commit()` and the revocation is cleared back to `type(uint64).max`, restoring full vesting eligibility. The beneficiary receives the original allocation despite the recorded revocation.
2. **Resurrected post-reveal.** After a successful `Vesting.reveal()`, `commitments[_hash]` is set to 0 (line 242). Calling `Vesting.commit()` again resets it to `type(uint64).max`, allowing a second `reveal()` with the same parameters and creating a duplicate vesting plan with the same beneficiary, allocation, cliff, duration, and salt.
3. **Frontrun of a redundant commit.** If two `Vesting.commit()` transactions for the same hash are submitted (e.g., by different managers, a meta-transaction relayer replay, or accidental re-submission), the beneficiary — who already possesses the preimage — can front-run the second `Vesting.commit()` with a `Vesting.reveal()`. The first commitment is consumed (set to 0), the second `Vesting.commit()` then recreates it (`type(uint64).max`), and the beneficiary calls `Vesting.reveal()` a second time. Two identical vesting plans are created from what was

intended as a single commitment, doubling the beneficiary's allocation.

While case 2 can also be reached via direct `Vesting.createVesting()` calls (so no extra power is gained), case 1 silently undoes a recorded revocation in a way that is not visible from the original `Revoke` event alone, and case 3 allows the beneficiary to exploit a duplicated commit transaction without any manager explicitly re-committing after a reveal.

Assets:

- ./Vesting.sol [https://github.com/corpus-io/tokenize.it-smart-contracts.git]

Status: Fixed

Classification

Impact Rate: 3/5

Likelihood Rate: 2/5

Exploitability: Dependent

Complexity: Simple

Severity: Low

Recommendations

Remediation: In `Vesting.commit()`, it is recommended to require that `commitments[_hash] == 0` before writing, e.g., `require(commitments[_hash] == 0, CommitmentAlreadyExists());`. Re-using a hash should require an explicit, separately-named function so the manager's intent is auditable.

Resolution: The issue was fixed in commit `73780ab`. A `require(commitments[_hash] == 0, CommitmentAlreadyExists())` check was added to `Vesting.commit()`, preventing overwriting of existing commitments including those that have been revoked or already revealed.

F-2026-16771 - Zero Price Allows Free Token Acquisition and Causes Division-by-Zero - Low

Description:

The `Crowdinvesting` contract does not validate that `priceMin` is non-zero when activating dynamic pricing via `Crowdinvesting._activateDynamicPricing()`. The only check applied is `_priceMin <= priceBase`, which always passes for `_priceMin = 0` since `priceBase` is guaranteed to be positive. When the `PriceLinear` oracle is configured with a falling price (`isRising == false`), its `Crowdinvesting.getPrice()` function explicitly returns 0 once the cumulative `change` exceeds `basePrice`:

```
return basePrice <= change ? 0 : basePrice - change;
```

When this occurs, `Crowdinvesting.getPrice()` returns 0 because `priceMin = 0` provides no floor:

```
function getPrice() public view returns (uint256) {
    if (address(priceOracle) != address(0)) {
        uint256 price = priceOracle.getPrice(priceBase);
        if (price > priceMax) {
            return priceMax;
        }
        if (price < priceMin) {
            return priceMin;
        }
        return price;
    }

    return priceBase;
}
```

The zero price propagates into two buy paths with different effects:

- `Crowdinvesting.buy()` – **free token acquisition:** The currency cost is computed as `Math.ceilDiv(_tokenAmount * getPrice(), 10 ** token.decimals())`. When `Crowdinvesting.getPrice()` returns 0, the product is 0 and `Math.ceilDiv(0, ...)` evaluates to 0. The buyer pays zero currency yet receives the full `_tokenAmount` of tokens via `Crowdinvesting._checkAndDeliver()`.
- `Crowdinvesting.onTokenTransfer()` – **division by zero revert:** The token amount is computed as `(_currencyAmount * 10 ** token.decimals()) / getPrice()`. When `Crowdinvesting.getPrice()` returns 0, this causes an

unconditional revert, making the ERC-677 `transferAndCall` buy path unusable.

An attacker can acquire tokens for free through the `buy()` function up to the `maxAmountPerBuyer` limit. Since there is no restriction on the number of addresses an attacker controls, they can repeat this across multiple accounts to drain up to `maxAmountOfTokenToBeSold` tokens from the contract at zero cost. This directly results in loss of funds for the token issuer, who receives no payment for the minted or transferred tokens. Additionally, the `Crowdinvesting.onTokenTransfer()` path becomes bricked, causing a denial of service for any user attempting to purchase tokens via the ERC-677 `transferAndCall()` mechanism.

Assets:

- ./Crowdinvesting.sol [https://github.com/corpus-io/tokenize.it-smart-contracts.git]

Status: Fixed

Classification

Impact Rate: 4/5

Likelihood Rate: 3/5

Exploitability: Dependent

Complexity: Simple

Severity: Low

Recommendations

Remediation: Add a zero-value check for `_priceMin` in `Crowdinvesting._activateDynamicPricing()` to prevent the price from ever reaching zero:

```
function _activateDynamicPricing(IPriceDynamic _priceOracle, uint256 _priceMi
n, uint256 _priceMax) internal {
    require(_priceMin != 0, ZeroPrice());
    ...
}
```

Additionally, consider adding a defensive check in `Crowdinvesting.getPrice()` to guarantee the returned value is always non-zero:

```
function getPrice() public view returns (uint256) {
    if (address(priceOracle) != address(0)) {
        uint256 price = priceOracle.getPrice(priceBase);
        if (price > priceMax) {
            return priceMax;
        }
        if (price < priceMin) {
            return priceMin;
        }
        require(price != 0, ZeroPrice());
        return price;
    }
    return priceBase;
}
```

Resolution: The issue was fixed in commit `3bdb683`. A `require(_priceMin != 0, ZeroPrice())` check was added to `Crowdinvesting._activateDynamicPricing()` and a `require(price != 0, ZeroPrice())` check was added to `Crowdinvesting.getPrice()` after receiving the oracle's return value, preventing zero-price token acquisition and division-by-zero scenarios.

F-2026-16772 - Missing Zero-Address Validation for _owner in PayoutBase May Lead to Loss of Administrative Control - Low

Description:

The `PayoutBase.__PayoutBase_init()` function calls `Ownable2StepUpgradeable._transferOwnership(_owner)` without verifying that `_owner` is not `address(0)`. The internal `Ownable2StepUpgradeable._transferOwnership()` function inherited from OpenZeppelin's `Ownable2StepUpgradeable` unconditionally sets the owner storage slot to the provided address without any zero-address guard — unlike the public `Ownable2StepUpgradeable.transferOwnership()`, which explicitly reverts on `address(0)`.

```
function __PayoutBase_init(
    address _owner,
    Token _token,
    IERC20 _currency,
    uint256 _pricePerToken,
    uint64 _lockedUntil
) internal onlyInitializing {
    __ReentrancyGuard_init();
    __Ownable2Step_init();
    _transferOwnership(_owner);
    token = _token;
    currency = _currency;
    pricePerToken = _pricePerToken;
    lockedUntil = _lockedUntil;
}
```

Both `Distribution.initialize()` and `Exit.initialize()` forward the caller-supplied `_arguments.owner` directly into `PayoutBase.__PayoutBase_init()` without any prior validation, propagating the issue to all contracts that inherit `PayoutBase`.

If `address(0)` is passed as the `_owner` during initialization — whether through a misconfigured deployment script, a front-end bug, or an incorrect factory call — the contract is permanently rendered ownerless. Since `PayoutBase` inherits `Ownable2StepUpgradeable`, recovering ownership requires the current owner to propose a new owner via `Ownable2StepUpgradeable.transferOwnership()`, which is gated by the `onlyOwner` modifier. With `owner() == address(0)`, no externally owned account or contract can satisfy this check, making recovery impossible.

All `onlyOwner`-gated functionality becomes permanently inaccessible:

- `PayoutBase.drain()` — the owner cannot recover remaining currency after `lockedUntil` elapses, locking funds in the contract indefinitely.
- `Distribution.reassign()` — the owner cannot redirect unclaimed distributions to correct recipients, e.g. in cases of lost keys.

Because these contracts are deployed as minimal proxy clones, the `initializer` modifier ensures `initialize()` can only be called once. There is no upgrade path or re-initialization mechanism to correct the owner after deployment.

Assets:

- common/PayoutBase.sol [https://github.com/corpus-io/tokenize.it-smart-contracts.git]

Status: Fixed

Classification

Impact Rate: 4/5

Likelihood Rate: 3/5

Exploitability: Dependent

Complexity: Simple

Severity: Low

Recommendations

Remediation: Add an explicit zero-address check for `_owner` in `PayoutBase.__PayoutBase_init()` before calling `Ownable2StepUpgradeable._transferOwnership()`:

```
function __PayoutBase_init(
    address _owner,
    Token _token,
    IERC20 _currency,
    uint256 _pricePerToken,
    uint64 _lockedUntil
) internal onlyInitializing {
    require(_owner != address(0), ZeroOwnerAddress());
    __ReentrancyGuard_init();
    __Ownable2Step_init();
    _transferOwnership(_owner);
    token = _token;
    currency = _currency;
    pricePerToken = _pricePerToken;
```

```
        lockedUntil = _lockedUntil;
    }
```

Resolution:

Fixed in `cff2c71`. An explicit zero-address check for `_owner` was added to `PayoutBase.__PayoutBase_init()` before calling `_transferOwnership(_owner)`. This prevents initialization with a zero-address owner, which would render the contract permanently ownerless;

```
function __PayoutBase_init(
    ...
) internal onlyInitializing {
    require(_owner != address(0), ZeroOwnerAddress());
    ...
}
```

F-2026-16482 - Single Step Ownership Model Enables Immediate Loss Of Administrative Control - Info

Description:

The `TokenSwapBase`, `Vesting` and `TimeLock` contracts use OpenZeppelin's `OwnableUpgradeable` access control mechanism, which implements ownership transfer in a single-step process. Administrative control is transferred immediately upon execution of `OwnableUpgradeable.transferOwnership()`, without requiring acceptance from the recipient address.

No validation is performed to ensure that the new owner is capable of interacting with privileged functionality. As a result, ownership may be assigned to an incorrect address, an incompatible contract, or a non-operational account.

If ownership is transferred to an unintended or non-functional address, access to owner-restricted functionality may become permanently unavailable. Affected operations include administrative configuration, market creation, registry management, and other privileged actions governed by ownership control.

Assets:

- ./Vesting.sol [https://github.com/corpus-io/tokenize.it-smart-contracts.git]
- ./common/TokenSwapBase.sol [https://github.com/corpus-io/tokenize.it-smart-contracts.git]
- ./TimeLock.sol [https://github.com/corpus-io/tokenize.it-smart-contracts.git]

Status: Fixed

Classification

Impact Rate: 1/5

Likelihood Rate: 5/5

Exploitability: Dependent

Complexity: Simple

Severity: Info

Recommendations

Remediation: It is recommended to replace the single-step ownership transfer mechanism with a two-step ownership model (`Ownable2StepUpgradeable`), where ownership transfer is completed only after explicit acceptance by the recipient address.

Resolution: The issue was fixed in commit `ba9ff90`. The `OwnableUpgradeable` access control contract was replaced with `Ownable2StepUpgradeable` in the `TokenSwapBase`, `TimeLock`, and `Vesting` contracts.

F-2026-16638 - Public Function Not Used Internally Can be Marked External - Info

Description:

There are some functions within the codebase that are currently marked as public, but are not invoked internally within the contract itself. Since these functions are only intended to be called externally, it should be marked as external instead of public.

Affected functions:

- `AllowList.initialize()`
- `CoinvestedPosition.buy()`
- `Crowdinvesting.buy()`
- `PriceLinear.getPrice()`
- `TimeLock.initialize()`
- `Token.initialize()`
- `TokenSwap.buy()`
- `TokenSwap.sell()`
- `Vesting.initialize()`
- `Vesting.releasable()`

In Solidity, functions that are only meant to be invoked by external actors, such as external contracts or users, should be designated as `external` rather than `public`. While both `public` and `external` functions are accessible from outside the contract, there are key differences. A `public` function can be called both from other functions inside the same contract and from outside transactions, while an `external` function can only be called from outside the contract. Specifically, `external` functions are more gas-efficient when called externally, as they do not require the extra internal checks that `public` functions do. Marking a function as `public` instead of `external` introduces unnecessary gas costs.

Assets:

- ./Crowdinvesting.sol [https://github.com/corpus-io/tokenize.it-smart-contracts.git]
- ./Vesting.sol [https://github.com/corpus-io/tokenize.it-smart-contracts.git]
- ./Token.sol [https://github.com/corpus-io/tokenize.it-smart-contracts.git]
- ./CoinvestedPosition.sol [https://github.com/corpus-io/tokenize.it-smart-contracts.git]
- ./PriceLinear.sol [https://github.com/corpus-io/tokenize.it-smart-contracts.git]
- ./TimeLock.sol [https://github.com/corpus-io/tokenize.it-smart-contracts.git]

- ./AllowList.sol [https://github.com/corpus-io/tokenize.it-smart-contracts.git]
- ./TokenSwap.sol [https://github.com/corpus-io/tokenize.it-smart-contracts.git]

Status: Fixed

Classification

Impact Rate: 1/5

Likelihood Rate: 5/5

Exploitability: Independent

Complexity: Simple

Severity: Info

Recommendations

Remediation: Consider changing visibility to `external` instead of `public` to enhance gas efficiency and reduce unnecessary exposure for the specified functions. This change ensures that the function is only callable from external sources, reducing internal contract complexity and improving security.

Resolution: The issue was fixed in commit `27ac75c`. The `initialize()` functions in `AllowList`, `Token`, `TimeLock`, and `Vesting` and the `buy()` / `sell()` / `getPrice()` / `releasable()` functions were changed from `public` to `external` visibility where they are not called internally.

F-2026-16639 - Lack of Event Emitting for Important State Changes - Info

Description:

Several functions within the **Tokenize.it** protocol perform meaningful state changes and token movements without emitting events, which hinders off-chain observability:

- `CoinvestedPosition.setCurrency()`
- `CoinvestedPosition._settle()`
- `CoinvestedPosition.claimDistribution()`
- `CoinvestedPosition.claimExit()`
- `Distribution.claim()`
- `Exit.claim()`
- `PriceLinear.updateParameters()`
- `PayoutBase.drain()`

The absence of an event makes it difficult to track admin-specific actions off-chain, which reduces transparency and complicates monitoring and auditing. It is recommended to emit events for this operation to ensure proper visibility, support off-chain indexing, and facilitate debugging, especially for critical administrative and user-facing fund transfers.

Assets:

- ./CoinvestedPosition.sol [https://github.com/corpus-io/tokenize.it-smart-contracts.git]
- ./Distribution.sol [https://github.com/corpus-io/tokenize.it-smart-contracts.git]
- ./Exit.sol [https://github.com/corpus-io/tokenize.it-smart-contracts.git]
- ./PriceLinear.sol [https://github.com/corpus-io/tokenize.it-smart-contracts.git]
- common/PayoutBase.sol [https://github.com/corpus-io/tokenize.it-smart-contracts.git]

Status: Fixed

Classification

Impact Rate: 1/5

Likelihood Rate: 5/5

Exploitability: Independent

Complexity: Simple

Severity: Info

Recommendations

Remediation: Consider emitting events within the mentioned affected functions to enhance transparency and ensure reliable off-chain tracking of key state changes. Emitting events not only improves monitoring of critical storage variable updates and key operation execution but also supports indexing services, facilitates auditing, and provides better visibility into administrative actions.

Resolution: The issue was fixed in commit `b0158b9`. Events were added for key state-changing operations: `CurrencyChanged` and `ExitClaimed` in `CoinvestedPosition`, `Claimed` in `Distribution` and `Exit`, `Drained` in `PayoutBase`, and `ParametersUpdated` in `PriceLinear`.

F-2026-16641 - Repeated Storage Reads of leadInvestors.length in a Loop Condition - Info

Description:

Within the `CoinvestedPosition._settle()` function, `leadInvestors.length` is read directly from storage in a loop condition on every iteration without being cached in a local variable:

```
function _settle(uint256 profit, IERC20 _currency) internal {
    require(address(_currency) != address(token), CurrencyEqualsToken());
    for (uint256 i = 0; i < leadInvestors.length; i++) {
        uint256 carryFraction = (uint256(leadInvestors[i].profitFraction) * p
rofit) / type(uint64).max;
        if (carryFraction != 0) {
            _currency.safeTransfer(leadInvestors[i].account, carryFraction);
        }
    }
    _currency.safeTransfer(receiver, _currency.balanceOf(address(this)));
}
```

Since `leadInvestors` is a storage array, each access to `leadInvestors.length` results in an `SLOAD` (cold: 2100 gas, warm: 100 gas) of the array length slot. This introduces unnecessary gas overhead. In particular, the loop repeatedly re-reads the same storage value despite it remaining constant throughout execution. Each redundant warm `SLOAD` after the first costs 100 gas, so with **N** lead investors the overhead is approximately **100 * (N – 1)** gas. `CoinvestedPosition._settle()` is widely called within the main contract functions `CoinvestedPosition.buy()`, `CoinvestedPosition.claimDistribution()`, and `CoinvestedPosition.claimExit()`, so this overhead is incurred on every settlement and dividend claim.

Assets:

- ./CoinvestedPosition.sol [https://github.com/corpus-io/tokenize.it-smart-contracts.git]

Status: Fixed

Classification

Impact Rate: 1/5

Likelihood Rate: 5/5

Exploitability: Independent

Complexity: Simple

Severity: Info

Recommendations

Remediation: Cache the `leadInvestors` array length in a local variable before the loop:

```
function _settle(uint256 profit, IERC20 _currency) internal {
    uint256 len = leadInvestors.length;
    for (uint256 i = 0; i < len; i++) {
        ...
    }
    ...
}
```

Resolution: The issue was fixed in commit `d224de1`. The `leadInvestors.length` value is now cached in a local variable `leadInvestorsLength` before the loop in `CoinvestedPosition._credit()`, avoiding repeated SLOAD operations on each iteration.

F-2026-16656 - Mismatched Cooldown Period Between Implementation and Invariants Breaks Assumptions - Info

Description:

The `Crowdinvesting` contract defines a cooldown period through the constant `delay`, which is set to `1 hours`. This value is reinforced in the implementation documentation, which states that after any parameter change, a 1-hour delay is enforced before the contract can be unpaused again.

```
/// @notice Minimum waiting time between pause or parameter change and unpaus
e.
/// @dev delay is calculated from parameter change to unpause.
uint256 public constant delay = 1 hours;
```

However, the protocol invariants documentation defines the same cooldown behavior differently, stating that each settings update restarts a cooldown period of **24 hours**. This creates a discrepancy between the documented invariants and the deployed implementation.

```
## Crowdinvesting.sol
...
- Each setting update (re-)starts the cool down period of 24h hours.
```

As a result, the effective cooldown period enforced on-chain is materially shorter than the duration described in the protocol invariants. Any operational, integration, or governance assumptions based on a 24-hour cooldown window are therefore invalid under the current implementation.

Assets:

- ./Crowdinvesting.sol [https://github.com/corpus-io/tokenize.it-smart-contracts.git]

Status: Fixed

Classification

Impact Rate: 1/5

Likelihood Rate: 5/5

Exploitability: Independent

Complexity: Simple

Severity: Info

Recommendations

Remediation: Decide which value reflects intended protocol behavior. Either extend the on-chain `delay` to `24 hours` to match `invariants.md`, or update `invariants.md` and any related documentation to reflect the 1-hour value. Additionally, unify the wording across `dev_overview.md`, `invariants.md`, and the contract's own NatSpec.

Resolution: The issue was fixed in commit `01eead7`. The NatSpec documentation in `Crowdinvesting` was corrected to accurately describe that the cooldown delay is calculated from the last parameter change to unpause (not from pause) and that the enforced value is 1 hour, matching the actual implementation behavior. The invariants documentation (`docs/invariants.md`) was updated to consistently reference the 1-hour cooldown period, resolving the discrepancy with the previously documented 24-hour value.

F-2026-16657 - Token Fee-Collector Compliance Bypass Applies Only to Mints Instead of All Transfers - Info

Description: The invariants document states that an address may receive tokens if it is the fee collector. However, the on-chain check `Token._checkIfAllowedToTransact(_to)` does not include any `feeCollector` exception:

```
// Token.sol
function _checkIfAllowedToTransact(address _address) internal view {
    require(requirements == 0 || hasRole(TRANSFERER_ROLE, _address) ||
                allowList.map(_address) & requirements == requirements,
            NotAllowedToTransact());
}
```

```
// invariants.md
- An address can only receive tokens at least one of these statements is true
:
  - The address fulfills the requirements as proven by the allowList.
  - The address has the TRANSFERER_ROLE.
  - The address is the feeCollector.
  - The address is the 0 address.
```

The "fee collector is always allowed" property is enforced implicitly only during minting, because `Token._beforeTokenTransfer()` skips the compliance check when `_from == address(0)`. A regular ERC20 transfer to a fee collector that does not satisfy `requirements` and lacks `TRANSFERER_ROLE` reverts.

```
function mint(address _to, uint256 _amount) external {
    if (!hasRole(MINTALLOWER_ROLE, _msgSender())) {
        require(mintingAllowance[_msgSender()] >= _amount, MintingAllowanceTo
oLow());
        mintingAllowance[_msgSender()] -= _amount;
    }
    // this check is executed here, because later minting of the buy amount c
an not be
    // differentiated from minting of the fee amount
    _checkIfAllowedToTransact(_to);
    _mint(_to, _amount);
    // collect fees
    uint256 fee = feeSettings.tokenFee(_amount, address(this));
    if (fee != 0) {
        // the fee collector is always allowed to receive tokens
        _mint(feeSettings.tokenFeeCollector(address(this)), fee);
```

```
    }
}
```

Assets:

- ./Token.sol [https://github.com/corpus-io/tokenize.it-smart-contracts.git]

Status: Fixed

Classification

Impact Rate: 1/5

Likelihood Rate: 5/5

Exploitability: Independent

Complexity: Simple

Severity: Info

Recommendations

Remediation: Either (a) update `docs/invariants.md` to scope the exception to mints only, or (b) introduce an explicit fee-collector exemption in `Token._checkIfAllowedToTransact()`, e.g. `_address == feeSettings.tokenFeeCollector(address(this))`.

Resolution: The issue was fixed in commit `773ec43`. The invariants documentation was updated to specify that the `feeCollector` address may only receive tokens when they are freshly minted (as part of the minting fee), not via arbitrary transfers. This reflects the existing implementation behavior.

F-2026-16741 - Missing State Cleanup After Fee Settings Acceptance - Info

Description:

After successful acceptance of a new fee settings contract, the active configuration is updated by assigning `feeSettings = suggestedFeeSettings`. However, the `suggestedFeeSettings` storage variable is not cleared after activation. As a result, the pending suggestion state remains populated even though the suggestion has already been accepted and promoted to the active configuration.

```
function suggestNewFeeSettings(IFeeSettingsV2 _feeSettings) external {
    require(_msgSender() == feeSettings.owner(), OnlyFeeSettingsOwner());
    _checkIfFeeSettingsImplementsInterface(_feeSettings);
    suggestedFeeSettings = _feeSettings;
    emit NewFeeSettingsSuggested(_feeSettings);
}

function acceptNewFeeSettings(IFeeSettingsV2 _feeSettings) external onlyRole(
DEFAULT_ADMIN_ROLE) {
    // after deployment, suggestedFeeSettings is 0x0. Therefore, this check i
s necessary, otherwise
    // the admin could accept 0x0 as new feeSettings. Checking that the sugge
stedFeeSettings is not
    // 0x0 would work, too, but this check is used in other places, too.
    _checkIfFeeSettingsImplementsInterface(_feeSettings);

    require(address(_feeSettings) == address(suggestedFeeSettings), OnlySugge
stedFeeSettings());
    feeSettings = suggestedFeeSettings;
    emit FeeSettingsChanged(_feeSettings);
}
```

This creates an inconsistent state representation where `Token.suggestedFeeSettings()` continues to return a valid contract address, despite no pending update existing.

The issue affects state transparency and introduces ambiguity for off-chain consumers, integrations, and monitoring systems that rely on `suggestedFeeSettings` to determine whether a configuration change is pending.

Under the current implementation, the persisted value remains indistinguishable from a genuinely pending suggestion until a new `Token.suggestNewFeeSettings()` call overwrites it. During this period, external

systems may incorrectly infer that an unaccepted fee settings update still exists.

Assets:

- ./Token.sol [https://github.com/corpus-io/tokenize.it-smart-contracts.git]

Status: Fixed

Classification

Impact Rate: 1/5

Likelihood Rate: 5/5

Exploitability: Independent

Complexity: Simple

Severity: Info

Recommendations

Remediation: The `suggestedFeeSettings` storage variable should be cleared immediately after successful acceptance.

State cleanup after activation ensures that the pending configuration state accurately reflects whether an unaccepted suggestion exists and preserves consistency between active and pending configuration tracking.

Resolution: The issue was fixed in commit `02aed53`. The `Token.acceptNewFeeSettings()` function now sets `suggestedFeeSettings = IFeeSettingsV2(address(0))` after accepting the new fee settings, clearing the stale suggestion and preventing it from being accepted again.

F-2026-16743 - Missing Self-Reassignment Validation - Info

Description:

The `Distribution._reassign()` function does not validate that the source and destination addresses are distinct. As a result, execution is permitted when `_from == _to`. In this scenario, the same `_amount` is added to both `paidOut[_from]` and `extraCredit[_to]`. Since both mappings reference the same address, the resulting state changes offset each other in the `_grossEligible` calculation. This causes the effective claimable balance to remain unchanged, producing no economic or functional effect. Despite having no operational impact, the transaction completes successfully, modifies storage, emits a `Reassigned` event, and consumes gas.

```
function reassign(address _from, address _to, uint256 _amount) external onlyO
wner {
    require(block.timestamp >= lockedUntil, ReassignmentNotYetAvailable());
    _reassign(_from, _to, _amount);
}

function _reassign(address _from, address _to, uint256 _amount) internal {
    require(_to != address(0), ZeroReceiverAddress());
    require(_amount > 0, ZeroAmount());
    require(_amount <= _grossEligible(_from), ReassignmentExceedsEligible());
    paidOut[_from] += _amount;
    extraCredit[_to] += _amount;
    emit Reassigned(_from, _to, _amount);
}
```

Assets:

- ./Distribution.sol [https://github.com/corpus-io/tokenize.it-smart-contracts.git]

Status: Fixed

Classification

Impact Rate: 1/5

Likelihood Rate: 5/5

Exploitability: Dependent

Complexity: Simple

Severity: Info

Recommendations

Remediation: It is recommended to add a `require(_from != _to, SelfReassignmentNotAllowed());` check at the beginning of `Distribution._reassign()`.

Resolution: The issue was fixed in commit `eb7222c`. A `require(_from != _to, SelfReassignmentNotAllowed())` check was added to `Distribution._reassign()`, preventing self-reassignment which would emit misleading events and waste gas on a no-op.

F-2026-16768 - Gas Inefficiency Due to Redundant Boolean Equality Check - Info

Description:

In the `Token` contract function `Token._checkIfFeeSettingsImplementsInterface()` contains direct boolean comparison:

```
// step 1: needs to return true if EIP165 is supported
require(_feeSettings.supportsInterface(0x01ffc9a7) == true, FeeSettingsInterfaceNotSupported());
// step 2: needs to return false if EIP165 is supported
require(_feeSettings.supportsInterface(0xffffffff) == false, FeeSettingsInterfaceNotSupported());
```

Using a direct boolean comparison is considered less gas-efficient in Solidity. The more conventional and preferred pattern is:

```
// step 1: needs to return true if EIP165 is supported
require(_feeSettings.supportsInterface(0x01ffc9a7), FeeSettingsInterfaceNotSupported());
// step 2: needs to return false if EIP165 is supported
require(!_feeSettings.supportsInterface(0xffffffff), FeeSettingsInterfaceNotSupported());
```

This does not introduce a functional vulnerability, as both patterns evaluate the same boolean result. However, direct comparison against `true` or `false` introduces unnecessary bytecode/gas overhead and reduces code readability compared to the standard Solidity boolean-check pattern.

Assets:

- ./Token.sol [https://github.com/corpus-io/tokenize.it-smart-contracts.git]

Status: Fixed

Classification

Impact Rate: 1/5

Likelihood Rate: 5/5

Exploitability: Independent

Complexity: Simple

Severity: Info

Recommendations

Remediation: Consider using Boolean variable directly instead of comparing it to `true` or `false`, to save gas and make the contract more efficient.

Resolution: The issue was fixed in commit `dfccaf4`. The redundant boolean comparisons `== true` and `== false` in `Token._checkIfFeeSettingsImplementsInterface()` were simplified to direct boolean expressions and negation (`!`).

F-2026-16769 - Redundant coolDownStart Assignment in activateDynamicPricing - Info

Description:

In `Crowdinvesting.activateDynamicPricing()`, `coolDownStart` is set to `block.timestamp` immediately after calling `Crowdinvesting._activateDynamicPricing()`. However, `Crowdinvesting._activateDynamicPricing()` already assigns `coolDownStart = block.timestamp`. This results in a redundant SSTORE to the same storage slot with the same value within the same transaction, wasting gas without any functional effect.

```
function activateDynamicPricing(
    IPriceDynamic _priceOracle,
    uint256 _priceMin,
    uint256 _priceMax
) external onlyOwner whenPaused {
    _activateDynamicPricing(_priceOracle, _priceMin, _priceMax);
    coolDownStart = block.timestamp;
}

function _activateDynamicPricing(IPriceDynamic _priceOracle, uint256 _priceMi
n, uint256 _priceMax) internal {
    require(address(_priceOracle) != address(0), ZeroPriceOracleAddress());
    priceOracle = _priceOracle;
    require(_priceMin <= priceBase, PriceMinExceedsPriceBase());
    priceMin = _priceMin;
    require(priceBase <= _priceMax, PriceMaxBelowPriceBase());
    priceMax = _priceMax;
    coolDownStart = block.timestamp;

    emit DynamicPricingActivated(address(_priceOracle), _priceMin, _priceMax)
;
}
```

The redundant SSTORE operation wastes approximately 100 gas per call to `Crowdinvesting.activateDynamicPricing()`, as the same storage slot is written with an identical value twice in the same transaction. While this has no effect on contract logic or security, it unnecessarily increases transaction costs for the contract owner.

Assets:

- ./Crowdinvesting.sol [https://github.com/corpus-io/tokenize.it-smart-contracts.git]

Status: Fixed

Classification

Impact Rate: 1/5

Likelihood Rate: 5/5

Exploitability: Independent

Complexity: Simple

Severity: Info

Recommendations

Remediation: Remove the redundant `coolDownStart = block.timestamp;` assignment in `Crowdinvesting.activateDynamicPricing()`, since `Crowdinvesting._activateDynamicPricing()` already sets it to the same value in the same transaction context.

Resolution: The issue was fixed in commit `9ce8477`. The redundant `coolDownStart = block.timestamp` assignment was removed from `Crowdinvesting.activateDynamicPricing()` since the same assignment is already performed inside `_activateDynamicPricing()` which is called by the public function.

F-2026-16770 - Missing Inherited Initializer Calls in Clone Contracts - Info

Description:

Multiple clone contracts inherit from OpenZeppelin's `PausableUpgradeable` and/or `ReentrancyGuardUpgradeable` but omit the corresponding `PausableUpgradeable.__Pausable_init()` and `ReentrancyGuardUpgradeable.__ReentrancyGuard_init()` calls in their `initialize()` functions. Specifically:

Contract	Inherits	Missing Initializer Call(s)
`Token.sol`	`PausableUpgradeable`	`PausableUpgradeable.__Pausable_init()`
`Vesting.sol`	`ReentrancyGuardUpgradeable`	`ReentrancyGuardUpgradeable.__ReentrancyGuard_init()`
`Crowdinvesting.sol`	`PausableUpgradeable`, `ReentrancyGuardUpgradeable`	`PausableUpgradeable.__Pausable_init()`, `ReentrancyGuardUpgradeable.__ReentrancyGuard_init()`
`TokenSwapBase.sol`	`PausableUpgradeable`, `ReentrancyGuardUpgradeable`	`PausableUpgradeable.__Pausable_init()`, `ReentrancyGuardUpgradeable.__ReentrancyGuard_init()`

OpenZeppelin's upgradeable contracts rely on explicit initializer calls instead of constructors. The `PausableUpgradeable.__Pausable_init()` function sets `_paused = false`, and `ReentrancyGuardUpgradeable.__ReentrancyGuard_init()` sets `_status = _NOT_ENTERED` (value `1`). When these calls are omitted, the corresponding storage slots remain at their default value.

For `PausableUpgradeable`, the default `bool` value of `false` coincidentally matches the intended unpaused state, so the contract functions correctly despite the missing call.

For `ReentrancyGuardUpgradeable`, the `_status` variable is `uint256` with `_NOT_ENTERED = 1` and `_ENTERED = 2`. Without `ReentrancyGuardUpgradeable.__ReentrancyGuard_init()`, `_status` defaults to `0`. While the `nonReentrant` modifier still prevents reentrant calls (since `0 != 2`), the first invocation of any `nonReentrant` function performs a cold `SSTORE` from `0 -> 2` instead of a warm `SSTORE` from `1 -> 2`. This costs an additional ~17,100 gas (cold storage write vs. warm storage modification) for the very first caller and prevents that caller from receiving the standard EIP-2929 gas refund on the `_status` reset.

The contracts currently function correctly due to favorable default values. However:

1. **Increased gas cost for the first caller:** In `Vesting`, `Crowdinvesting`, and `TokenSwapBase` (and its descendants), the first user to call a `nonReentrant`

function pays approximately 17,100 extra gas due to cold storage initialization of `_status`.

2. **Violation of the OpenZeppelin initialization pattern:** Skipping initializer calls deviates from the documented and expected usage of upgradeable contracts instances. This creates a fragile dependency on implementation details (default storage values) that could break if OpenZeppelin changes the internal initialization logic in future versions.
3. **Upgrade risk:** If the contracts are upgraded to a new OpenZeppelin version where the initializers set different default values or perform additional setup, the omission could introduce silent bugs.

Assets:

- ./Crowdinvesting.sol [https://github.com/corpus-io/tokenize.it-smart-contracts.git]
- ./Vesting.sol [https://github.com/corpus-io/tokenize.it-smart-contracts.git]
- ./Token.sol [https://github.com/corpus-io/tokenize.it-smart-contracts.git]
- ./common/TokenSwapBase.sol [https://github.com/corpus-io/tokenize.it-smart-contracts.git]

Status: Fixed

Classification

Impact Rate: 1/5

Likelihood Rate: 5/5

Exploitability: Independent

Complexity: Simple

Severity: Info

Recommendations

Remediation: Add the missing initializer calls in each affected contract's `initialize` function:

- `Token.sol`: add `PausableUpgradeable.__Pausable_init()`
- `Vesting.sol`: add `ReentrancyGuardUpgradeable.__ReentrancyGuard_init()`
- `Crowdinvesting.sol`: add `ReentrancyGuardUpgradeable.__ReentrancyGuard_init()` and `PausableUpgradeable.__Pausable_init()`
- `TokenSwapBase.sol`: add `ReentrancyGuardUpgradeable.__ReentrancyGuard_init()` and `PausableUpgradeable.__Pausable_init()`

This ensures proper initialization of all inherited state, eliminates the extra gas cost on first use of `nonReentrant`, and maintains forward compatibility with future OpenZeppelin upgrades.

Resolution: The issue was fixed in commit `35cd816`. Explicit `__Pausable_init()` and `__ReentrancyGuard_init()` initializer calls were added to `Crowdinvesting.initialize()`, `TokenSwapBase._initializeBase()`, and `Token.initialize()`, and `__ReentrancyGuard_init()` was added to `Vesting.initialize()`.

Disclaimers

Hacken Disclaimer

The smart contracts given for audit have been analyzed based on best industry practices at the time of the writing of this report, with cybersecurity vulnerabilities and issues in smart contract source code, the details of which are disclosed in this report (Source Code); the Source Code compilation, deployment, and functionality (performing the intended functions).

The report contains no statements or warranties on the identification of all vulnerabilities and security of the code. The report covers the code submitted and reviewed, so it may not be relevant after any modifications. Do not consider this report as a final and sufficient assessment regarding the utility and safety of the code, bug-free status, or any other contract statements.

While we have done our best in conducting the analysis and producing this report, it is important to note that you should not rely on this report only — we recommend proceeding with several independent audits and a public bug bounty program to ensure the security of smart contracts.

English is the original language of the report. The Consultant is not responsible for the correctness of the translated versions.

As part of Hacken's ongoing quality assurance process, we may conduct re-audits of select projects. These re-audits are performed independently from the original audit and are intended solely for internal quality control and improvement. Updated reports resulting from such re-audits will be shared privately with the respective clients and may be published on the Hacken website only with their explicit consent.
The sole authoritative source for finalized and most up-to-date versions of all reports remains the Audits section at https://hacken.io/audits/.

Technical Disclaimer

Smart contracts are deployed and executed on a blockchain platform. The platform, its programming language, and other software related to the smart contract can have vulnerabilities that can lead to hacks. Thus, the Consultant cannot guarantee the explicit security of the audited smart contracts.

Appendix 1. Definitions

Severities

When auditing smart contracts, Hacken is using a risk-based approach that considers **Likelihood**, **Impact**, **Exploitability** and **Complexity** metrics to evaluate findings and score severities.

Reference on how risk scoring is done is available through the repository in our Github organization:

hknio/severity-formula

Severity	Description
Critical	Critical vulnerabilities are usually straightforward to exploit and can lead to the loss of user funds or contract state manipulation.
High	High vulnerabilities are usually harder to exploit, requiring specific conditions, or have a more limited scope, but can still lead to the loss of user funds or contract state manipulation.
Medium	Medium vulnerabilities are usually limited to state manipulations and, in most cases, cannot lead to asset loss. Contradictions and requirements violations. Major deviations from best practices are also in this category.
Low	Major deviations from best practices or major Gas inefficiency. These issues will not have a significant impact on code execution.

Potential Risks

The "Potential Risks" section identifies issues that are not direct security vulnerabilities but could still affect the project's performance, reliability, or user trust. These risks arise from design choices, architectural decisions, or operational practices that, while not immediately exploitable, may lead to problems under certain conditions. Additionally, potential risks can impact the quality of the audit itself, as they may involve external factors or components beyond the scope of the audit, leading to incomplete assessments or oversight of key areas. This section aims to provide a broader perspective on factors that could affect the project's long-term security, functionality, and the comprehensiveness of the audit findings.

Appendix 2. Scope

The scope of the project includes the following smart contracts from the provided repository:

Scope Details	
Repository	https://github.com/corpus-io/tokenize.it-smart-contracts.git
Commit	4414c631a341dfed79611d040ae0f5aa9801243f
Final Commit	52b0322fb566c7143d09c23b7bd30f2e092e0691
Whitepaper	-
Requirements	https://github.com/corpus-io/tokenize.it-smart-contracts/tree/4414c631a341dfed79611d040ae0f5aa9801243f/docs
Technical Requirements	https://github.com/corpus-io/tokenize.it-smart-contracts/blob/4414c631a341dfed79611d040ae0f5aa9801243f/README.md

Asset	Type
./AllowList.sol [https://github.com/corpus-io/tokenize.it-smart-contracts.git]	Smart Contract
./CoinvestedPosition.sol [https://github.com/corpus-io/tokenize.it-smart-contracts.git]	Smart Contract
./common/TokenSwapBase.sol [https://github.com/corpus-io/tokenize.it-smart-contracts.git]	Smart Contract
./Crowdinvesting.sol [https://github.com/corpus-io/tokenize.it-smart-contracts.git]	Smart Contract
./Distribution.sol [https://github.com/corpus-io/tokenize.it-smart-contracts.git]	Smart Contract
./Exit.sol [https://github.com/corpus-io/tokenize.it-smart-contracts.git]	Smart Contract
./FeeSettings.sol [https://github.com/corpus-io/tokenize.it-smart-contracts.git]	Smart Contract
./GlobalTokenExitRegistry.sol [https://github.com/corpus-io/tokenize.it-smart-contracts.git]	Smart Contract
./PriceLinear.sol [https://github.com/corpus-io/tokenize.it-smart-contracts.git]	Smart Contract
./PrivateOffer.sol [https://github.com/corpus-io/tokenize.it-smart-contracts.git]	Smart Contract
./TimeLock.sol [https://github.com/corpus-io/tokenize.it-smart-contracts.git]	Smart Contract
./Token.sol [https://github.com/corpus-io/tokenize.it-smart-contracts.git]	Smart Contract
./TokenSwap.sol [https://github.com/corpus-io/tokenize.it-smart-contracts.git]	Smart Contract

Asset	Type
./Vesting.sol [https://github.com/corpus-io/tokenize.it-smart-contracts.git]	Smart Contract
common/PayoutBase.sol [https://github.com/corpus-io/tokenize.it-smart-contracts.git]	Smart Contract

Appendix 3. Additional Valuables

Additional Recommendations

The smart contracts in the scope of this audit could benefit from the introduction of automatic emergency actions for critical activities, such as unauthorized operations like ownership changes or proxy upgrades, as well as unexpected fund manipulations, including large withdrawals or minting events. Adding such mechanisms would enable the protocol to react automatically to unusual activity, ensuring that the contract remains secure and functions as intended.

To improve functionality, these emergency actions could be designed to trigger under specific conditions, such as:

- Detecting changes to ownership or critical permissions.
- Monitoring large or unexpected transactions and minting events.
- Pausing operations when irregularities are identified.

These enhancements would provide an added layer of security, making the contract more robust and better equipped to handle unexpected situations while maintaining smooth operations.

Frameworks and Methodologies

This security assessment was conducted in alignment with recognised penetration testing standards, methodologies and guidelines, including the NIST SP 800-115 – Technical Guide to Information Security Testing and Assessment, and the Penetration Testing Execution Standard (PTES), These assets provide a structured foundation for planning, executing, and documenting technical evaluations such as vulnerability assessments, exploitation activities, and security code reviews. Hacken's internal penetration testing methodology extends these principles to Web2 and Web3 environments to ensure consistency, repeatability, and verifiable outcomes.